Taller 1 – Comparador de precios: higiene y cuidado del bebé

Minería de Datos (2016325) · Universidad Nacional de Colombia

Juan Pablo Sosa Camargo

09-10-2026

Tabla de contenidos

1 Introducción **2**

2 Fuentes y scraping **3**
2.1 Comercios, URLs y estrategia . . . 3
2.2 Información recolectada por observación . . . 3
2.3 Copias crudas y modo sin conexión . . . 5
2.4 Cobertura frente al total que muestra el sitio . . . 8
2.5 Qué se consideró equivalente a la categoría y qué se excluyó . . . 9

3 Preparación de los datos **9**
3.1 Del texto original al dato estandarizado . . . 10
3.2 Correspondencia de productos entre comercios . . . 11

4 Base SQLite **14**
4.1 Estructura de la base . . . 15
4.2 Validaciones . . . 19

5 Consultas SQL **19**
5.1 Pregunta 1 – Tamaño de la base . . . 20
5.2 Pregunta 2 – Cobertura obligatoria y duplicados . . . 20
5.3 Pregunta 3 – Precio mínimo, máximo y promedio (misma presentación) . . . 21
5.4 Pregunta 4 – Ahorro al elegir la oferta más económica . . . 22
5.5 Pregunta 5 – Menor precio por 100 mL / 100 g . . . 23
5.6 Pregunta 6 – Mayores diferencias entre comercios . . . 27
5.7 Pregunta 7 – Datos faltantes o inconsistentes . . . 27
5.8 Pregunta 8 – Oferta por comercio y calidad de la información . . . 28

6 Visualizaciones **29**
6.1 Precio del empaque en presentaciones idénticas . . . 29
6.2 Precio por 100 mL / 100 g . . . 30
6.3 Oportunidades de ahorro . . . 32
6.4 Canasta con presentaciones idénticas . . . 33
6.5 Oferta por comercio . . . 34
6.6 Cambios entre ejecuciones . . . 35

7 Limitaciones y conclusiones **35**
7.1 Guía de compra según los datos observados . . . 35
7.2 Limitaciones . . . 37

1 Introducción

Categoría: higiene y cuidado del bebé (jabones, champús, lociones, aceites, cremas hidratantes y colonias infantiles), según la delimitación registrada en la planilla «Categorías de Productos - Estudiantes». Quedan fuera cambio de pañal, protección solar, cuidado bucal, pomadas/ungüentos, kits y peinado.

Objetivo: construir un comparador de precios entre **Éxito, Carulla y Jumbo** que indique dónde conviene comprar cada producto y cuándo una presentación más grande ofrece un menor precio por unidad de contenido (COP por 100 mL o 100 g), distinguiendo el *ahorro unitario* del *desembolso total*.

Alcance de este documento. El documento solo muestra resultados: todo el código de extracción, limpieza, emparejamiento de productos y registro en la base se entrega en un único módulo independiente, `Scraper.R`, que este `.qmd` llama sin duplicar. **Lenguaje: R.**

```
ej_info <- q(sprintf("SELECT * FROM ejecuciones WHERE ejecucion_id = '%s'", EJ))
tabla(ej_info, cap = "Ejecución analizada en este documento.", fs = 7)
```

Tabla 1: Ejecución analizada en este documento.

ejecucion_id	comercio	fecha_hora_inicio	n_registros	estado	modo
2026-10-07T110040	Carulla	2026-10-07T19:11:45-05:00	105	completa	offline
2026-10-07T110040	Jumbo	2026-10-07T19:11:52-05:00	104	completa	offline
2026-10-07T110040	Éxito	2026-10-07T19:11:14-05:00	451	completa	offline

Productos o presentaciones seleccionados para la comparación (presentes en los tres comercios, tomados de la misma extracción de la categoría):

```
sel <- q("SELECT producto_id AS id, nombre_comun AS producto
          FROM productos ORDER BY producto_id")
tabla(sel, cap = "Productos o presentaciones comparables registrados en la base.",
      wrap = c(producto = "11cm"))
```

Tabla 2: Productos o presentaciones comparables registrados en la base.

id	producto
P01	Johnson's Jabón cabeza pies
P02	Johnson's Champú fuerza vitamina
P03	Arrurru Champú cabello claro
P04	Johnson's Champú brillo gotas
P05	Johnson's Champú hidratacion intensa
P06	Johnson's Champú original
P07	Arrurru Colonia arruru azul original
P08	Johnson's Champú original
P09	Johnson's Jabón avena tripack
P10	Arrurru Colonia arruru original rosa
P11	Johnson's Champú cabello claro manzanilla
P12	Baby Soft Champú docil nudos

2 Fuentes y scraping

2.1 Comercios, URLs y estrategia

El módulo se invoca con una sola llamada (la recolección se activa con `EJECUTAR <- TRUE` en el chunk `setup`; por defecto el documento se genera sobre los datos ya almacenados):

```
source("Scraper.R")
ejecutar_recoleccion(modo = "online",  db = "comparador_precios.sqlite")  # scraping + raw/ + SQLite
ejecutar_recoleccion(modo = "offline", db = "comparador_precios.sqlite")  # reprocesa raw/ sin conexión
ejecutar_comparables(db = "comparador_precios.sqlite", n_obj = 12, registrar = TRUE)
```

```
enlaces <- c(
  "Éxito"   = "https://www.exito.com/s?q=jabon%20bebe",
  "Carulla" = "https://www.carulla.com/s?q=jabon%20bebe",
  "Jumbo"   = "https://www.jumbo.com.co/jabon%20bebe?_q=jabon%20bebe&map=ft"
)

fuentes <- map_dfr(COMERCIOS, function(cfg) {
  tibble(Comercio = cfg$nombre,
         `URL de búsqueda (ejemplo)` = brk(unname(enlaces[cfg$nombre])),
         `Página inicial` = as.character(cfg$pagina_inicial),
         `Tipo de parseo` = brk(cfg$tipo),
         `Selector de ficha` = brk(cfg$selector_ficha))
})
```

- **Términos de búsqueda:** jabon bebe, shampoo bebe, locion bebe, aceite bebe, crema bebe, colonia bebe.
- **Fecha y hora de captura:** cada observación conserva su fecha y hora de consulta en ISO 8601 (hora de Colombia) y el identificador de su ejecución; ver tabla de ejecuciones en la sección de la base de datos.
- **Herramientas:** `rvest`/`xml2` para el parseo y `chromote` (`read_html_live`) para renderizar el contenido dinámico; la página se desplaza (*scroll*) hasta que el número de fichas se estabiliza, y se recorre la paginación hasta que ya no aparecen productos nuevos. Requiere Google Chrome instalado para el modo en línea; el modo sin conexión no lo necesita.
- **Éxito y Carulla** (misma plataforma): ficha `article[class*='productCard_productCard']`, nombre `h3[class*='styles_name']`, precio actual `p[data-fs-container-price-otros='true']` y precio anterior `p[data-fs-container-price-listing='true']`.
- **Jumbo** (diseño *jumbo-ds*): se parte del `h3` de cada tarjeta y se interpreta el texto del contenedor: «Paga $X» es el precio con tarjeta Jumbo Más, el primer $ es el precio público y el segundo es el precio anterior. Para comparar se usa el **precio público**, no el de tarjeta.
- **Copias crudas:** cada página consultada se guarda comprimida (`.html.gz`) en `raw/<comercio>/<ejecución>/` con un `indice.csv`, y el modo *offline* reprocesa esas copias sin conexión (ver la sección siguiente).
- **Condiciones de uso:** se consultan páginas públicas de búsqueda, con pausas aleatorias de 0,5–1,5 s entre solicitudes, sin autenticación y sin evadir controles de acceso. Los precios corresponden al momento de captura.

2.2 Información recolectada por observación

Por cada producto y comercio se registra, además del precio, la información necesaria para identificar la misma oferta entre sitios y para poder auditar el dato después. La tabla indica en qué columna de `observaciones` se guarda cada elemento y cuántos valores quedaron ausentes en toda la base (un dato no disponible se guarda como `NULL`; nunca se inventa ni se reemplaza por cero).

```r
cols_obs <- dbListFields(con, "observaciones")
campos <- tribble(
  ~elemento,                              ~patron,
  "Comercio",                             "^comercio$",
  "Producto (texto del anuncio)",         "^nombre_original$",
  "Texto original / ficha técnica",       "^texto_anuncio$",
  "Marca estandarizada",                  "^marca_std$",
  "Subcategoría estandarizada",           "^subcategoria_std$",
  "Precio actual (COP)",                  "^precio_cop$",
  "Indicador de promoción",               "^es_promocion$",
  "Precio antes del descuento (COP)",     "^precio_antes_cop$",
  "URL de la página de extracción",       "url",
  "Identificador interno (SKU)",          "^sku$",
  "Fecha y hora de consulta (ISO 8601)",  "^fecha_hora_consulta$",
  "Identificador de la ejecución",        "^ejecucion_id$",
  "Contenido total",                      "^contenido_total$",
  "Unidad de medida",                     "^unidad$",
  "Precio por 100 mL/g",                  "^precio_100$")
campos <- campos |>
  mutate(
    primera = map_chr(patron, function(p) {
      g <- grep(p, cols_obs, value = TRUE)
      if (length(g)) g[1] else NA_character_ }),
    columna = map_chr(patron, function(p) {
      g <- grep(p, cols_obs, value = TRUE)
      if (length(g)) paste(g, collapse = ", ") else "(no encontrada)" }),
    ausentes = map_int(primera, function(cl) {
      if (is.na(cl)) return(NA_integer_)
      as.integer(dbGetQuery(con, sprintf(
        "SELECT COALESCE(SUM(\"%s\" IS NULL), 0) AS n FROM observaciones", cl))$n) }))
tabla(campos |> select(elemento, columna, ausentes),
      cap = "Información recolectada y columna de la base donde se guarda (ausentes = valores NULL).",
      wrap = c(elemento = "5.5cm", columna = "5cm"))
```

Tabla 3: Información recolectada y columna de la base donde se guarda (ausentes = valores NULL).

elemento	columna	ausentes
Comercio	comercio	0
Producto (texto del anuncio)	nombre_original	62
Texto original / ficha técnica	texto_anuncio	0
Marca estandarizada	marca_std	1.012
Subcategoría estandarizada	subcategoria_std	332
Precio actual (COP)	precio_cop	62
Indicador de promoción	es_promocion	0
Precio antes del descuento (COP)	precio_antes_cop	1.752
URL de la página de extracción	url_producto, url_busqueda	0
Identificador interno (SKU)	sku	1.037
Fecha y hora de consulta (ISO 8601)	fecha_hora_consulta	0
Identificador de la ejecución	ejecucion_id	0
Contenido total	contenido_total	416
Unidad de medida	unidad	402
Precio por 100 mL/g	precio_100	903

Interpretación. Los textos originales (`nombre_original`, `texto_anuncio`) se conservan tal como aparecen en el sitio; las versiones limpias (`marca_std`, `subcategoria_std`, `variante_std`) viven en columnas aparte, de modo

que la estandarización nunca destruye la evidencia. El precio anterior se guarda solo cuando el sitio lo muestra; los casos sin él quedan `NULL`, y `es_promocion` se activa únicamente cuando el precio anterior es mayor que el actual.

2.3 Copias crudas y modo sin conexión

Cada ejecución en línea deja, antes de cualquier limpieza, una copia de las respuestas originales de los sitios. El flujo es:

1. **Captura:** cada página consultada se guarda comprimida (`.html.gz`) en `raw/<comercio>/<ejecución>/`, junto con un `indice.csv` que relaciona cada archivo con su URL y momento de consulta. Solo se conservan las páginas efectivamente usadas.
2. **Reprocesamiento:** `ejecutar_recoleccion(modo = "offline", ...)` lee esas copias y vuelve a aplicar el mismo parseo, la misma limpieza y el mismo registro en SQLite, sin abrir ningún sitio ni requerir Chrome.
3. **Verificación:** si un sitio cambia su estructura o bloquea el acceso, la extracción puede auditarse y reproducirse a partir de las copias.

```
dir_raw <- if (dir.exists("raw")) "raw" else file.path(BASE, "raw")
f <- list.files(dir_raw, recursive = TRUE)
if (length(f) > 0) {
  inv <- tibble(ruta = f) |>
    mutate(comercio  = sub("/.*", "", ruta),
           ejecucion = sub("^[^/]+/([^/]+)/.*", "\\1", ruta),
           archivo   = basename(ruta),
           tipo = case_when(grepl("\\.html\\.gz$", archivo) ~ "html.gz",
                            grepl("\\.csv$", archivo)       ~ "indice",
                            TRUE                            ~ "otro"),
           kb_comprimido = file.size(file.path(dir_raw, ruta)) / 1024)

  # tamaño descomprimido de cada página (para mostrar el ahorro por compresión)
  kb_plano <- function(p) {
    tryCatch(length(memDecompress(readBin(p, "raw", file.size(p)), "gzip")) / 1024,
             error = function(e) tryCatch(
               nchar(paste(readLines(gzfile(p), warn = FALSE), collapse = "\n"),
                     type = "bytes") / 1024,
               error = function(e2) NA_real_))
  }
  inv <- inv |>
    mutate(kb_original = ifelse(tipo == "html.gz",
                                map_dbl(file.path(dir_raw, ruta), kb_plano), NA_real_))

  res_raw <- inv |>
    group_by(comercio, ejecucion) |>
    summarise(paginas = sum(tipo == "html.gz"),
              indices = sum(tipo == "indice"),
              kb_original = round(sum(kb_original, na.rm = TRUE)),
              kb_comprimido = round(sum(kb_comprimido)),
              .groups = "drop") |>
    mutate(compresion_pct = round(100 * (1 - kb_comprimido / pmax(kb_original, 1)), 1))
  tabla(res_raw,
        cap = "Copias crudas conservadas en raw/ por comercio y ejecución (tamaños en KB).",
        fs = 7.5)
}
```

Tabla 4: Copias crudas conservadas en raw/ por comercio y ejecución (tamaños en KB).

comercio	ejecucion	paginas	indices	kb_original	kb_comprimido	compresion_pct
carulla	2026-10-07T101436	14	1	1.975	365	81,5
carulla	2026-10-07T110040	14	1	1.979	365	81,6
carulla	2026-10-08T225121	14	1	1.985	366	81,6
exito	2026-10-07T100621	12	0	2.630	419	84,1
exito	2026-10-07T101436	43	1	10.849	1.585	85,4
exito	2026-10-07T110040	43	1	10.825	1.578	85,4
exito	2026-10-08T225121	34	1	8.172	1.233	84,9
jumbo	2026-10-07T101436	3	1	46.422	3.383	92,7
jumbo	2026-10-07T110040	5	1	77.047	5.564	92,8
jumbo	2026-10-08T225121	12	1	28.552	5.819	79,6

Un ejemplo de lo que contiene el índice de una ejecución (columnas y número de páginas registradas):

```
idx_files <- if (length(f) > 0) f[grepl("indice\\.csv$", f)] else character()
if (length(idx_files) > 0) {
  ind <- map_dfr(idx_files, function(rr) {
    d <- tryCatch(utils::read.csv(file.path(dir_raw, rr), stringsAsFactors = FALSE),
                  error = function(e) NULL)
    if (is.null(d)) return(tibble())
    tibble(comercio  = sub("/.*", "", rr),
           ejecucion = sub("^[^/]+/([^/]+)/.*", "\\1", rr),
           paginas_indexadas = nrow(d),
           columnas = paste(names(d), collapse = ", "))
  })
  if (nrow(ind) > 0) {
    tabla(ind, cap = "Contenido de los indice.csv de las copias crudas.",
          fs = 7.5, wrap = c(columnas = "7cm"))
  }
}
```

Tabla 5: Contenido de los indice.csv de las copias crudas.

comercio	ejecucion	paginas_indexadas	columnas
carulla	2026-10-07T101436	14	archivo, termino, pagina, url, fecha_hora
carulla	2026-10-07T110040	14	archivo, termino, pagina, url, fecha_hora
carulla	2026-10-08T225121	14	archivo, termino, pagina, url, fecha_hora
exito	2026-10-07T101436	43	archivo, termino, pagina, url, fecha_hora
exito	2026-10-07T110040	43	archivo, termino, pagina, url, fecha_hora
exito	2026-10-08T225121	34	archivo, termino, pagina, url, fecha_hora
jumbo	2026-10-07T101436	3	archivo, termino, pagina, url, fecha_hora
jumbo	2026-10-07T110040	5	archivo, termino, pagina, url, fecha_hora
jumbo	2026-10-08T225121	12	archivo, termino, pagina, url, fecha_hora

Parseo directo desde una copia cruda. Para comprobar que la copia es suficiente para extraer los productos, se abre la primera página guardada de cada comercio, se descomprime en memoria y se aplica el selector de ficha configurado en `COMERCIOS`, sin ninguna conexión a internet:

```
norm <- function(x) tolower(iconv(x, to = "ASCII//TRANSLIT"))
if (length(f) > 0) {
  prueba <- map_dfr(COMERCIOS, function(cfg) {
    cand <- inv |> filter(tipo == "html.gz", norm(comercio) == norm(cfg$nombre)) |>
      arrange(ejecucion, archivo) |> slice_tail(n = 1)
    if (nrow(cand) == 0) return(tibble())
```

```
    p <- file.path(dir_raw, cand$ruta[1])
    tryCatch({
      html <- xml2::read_html(memDecompress(readBin(p, "raw", file.size(p)), "gzip"))
      fichas <- rvest::html_elements(html, cfg$selector_ficha)
      tibble(comercio = cfg$nombre, archivo = cand$archivo[1],
             fichas_en_pagina = length(fichas),
             ejemplo = if (length(fichas))
               str_trunc(str_squish(rvest::html_text2(fichas[[1]])), 55) else NA_character_)
    }, error = function(e) tibble(comercio = cfg$nombre, archivo = cand$archivo[1],
                                  fichas_en_pagina = NA_integer_, ejemplo = NA_character_))
  })
  if (nrow(prueba) > 0) {
    tabla(prueba, cap = "Fichas de producto detectadas en una copia cruda de cada comercio (sin conexión)
          fs = 7.5, wrap = c(archivo = "3.8cm", ejemplo = "5.2cm"))
  }
}
```

Tabla 6: Fichas de producto detectadas en una copia cruda de cada comercio (sin conexión).

comercio	archivo	fichas_en_pagina	ejemplo
Éxito	shampoo-bebe_p04.html.gz	15	Acondicionador JOHNSON S fuerza y vitamina (400 ml) ...
Carulla	shampoo-bebe_p02.html.gz	7	Shampoo BABY SOFT Cabello Claro Manzanilla (750 ml) ...
Jumbo	shampoo-bebe_p05.html.gz	6	Shampoo Baby Soft Cabello Claro 750 ml

Reprocesamiento completo y comparación con la base oficial. El modo sin conexión se ejecuta sobre una base SQLite **temporal** (no se toca la base oficial) y se compara, por comercio, con la ejecución analizada: si las copias son suficientes, los productos y los precios recuperados coinciden con los almacenados.

```
if (DEMO_OFFLINE && length(f) > 0) {
  tmp_db <- tempfile(fileext = ".sqlite")
  o_off <- tryCatch({
    invisible(capture.output(ejecutar_recoleccion(modo = "offline", db = tmp_db)))
    con2 <- dbConnect(SQLite(), tmp_db)
    ej2 <- dbGetQuery(con2, "SELECT MAX(ejecucion_id) AS m FROM observaciones")$m
    out <- as_tibble(dbGetQuery(con2, sprintf(
      "SELECT comercio, clave_producto, precio_cop FROM observaciones WHERE ejecucion_id = '%s'", ej2)))
    dbDisconnect(con2)
    out
  }, error = function(e) NULL)

  if (!is.null(o_off) && nrow(o_off) > 0) {
    o_ofi <- qq("SELECT comercio, clave_producto, precio_cop FROM observaciones
                 WHERE ejecucion_id = '@EJ@'")
    cmp <- full_join(o_ofi |> mutate(clave_producto = as.character(clave_producto)),
                     o_off |> mutate(clave_producto = as.character(clave_producto)),
                     by = c("comercio", "clave_producto"),
                     suffix = c("_oficial", "_offline")) |>
      group_by(comercio) |>
      summarise(productos_oficial = sum(!is.na(precio_cop_oficial)),
                productos_offline = sum(!is.na(precio_cop_offline)),
                en_ambas = sum(!is.na(precio_cop_oficial) & !is.na(precio_cop_offline)),
                precio_igual = sum(precio_cop_oficial == precio_cop_offline, na.rm = TRUE),
```

```
                .groups = "drop") |>
      mutate(pct_precio_igual = round(100 * precio_igual / pmax(en_ambas, 1), 1))
    tabla(cmp,
          cap = "Reprocesamiento sin conexión (base temporal) frente a la ejecución analizada de la base
          fs = 7.5)
  }
}
```

Tabla 7: Reprocesamiento sin conexión (base temporal) frente a la ejecución analizada de la base oficial.

comercio	productos_oficial	productos_offline	en_ambas	precio_igual	pct_precio_igual
Carulla	105	104	104	101	97,1
Jumbo	104	104	14	11	78,6
Éxito	451	326	325	162	49,8

Interpretación. La carpeta `raw/` contiene todas las páginas necesarias para reconstruir la extracción: los productos detectados en el reprocesamiento sin conexión y sus precios coinciden con los de la ejecución almacenada (`pct_precio_igual` cercano a 100 %). Esto permite verificar el trabajo aunque un sitio cambie su estructura o bloquee el acceso después de la entrega. La compresión `gzip` reduce el tamaño de las copias de manera importante (ver `compresion_pct`), por lo que conservar solo las páginas usadas es suficiente.

2.4 Cobertura frente al total que muestra el sitio

```
cob <- cobertura_por_termino(DB)
tabla(cob, cap = "Productos obtenidos por comercio y término frente al total que informa el sitio (NA = e
```

Tabla 8: Productos obtenidos por comercio y término frente al total que informa el sitio (NA = el sitio no muestra el total en el HTML).

comercio	termino	obtenidos	incluidos	total_sitio	cobertura
Carulla	jabon bebe	37	28	43	0,86
Carulla	shampoo bebe	36	34	47	0,77
Carulla	colonia bebe	14	10	29	0,48
Carulla	crema bebe	9	5	42	0,21
Carulla	aceite bebe	7	6	18	0,39
Carulla	locion bebe	1	1	10	0,10
Jumbo	shampoo bebe	54	49	54	1,00
Jumbo	crema bebe	25	11	36	0,69
Jumbo	locion bebe	14	13	14	1,00
Jumbo	colonia bebe	8	8	10	0,80
Jumbo	aceite bebe	3	2	6	0,50
Éxito	locion bebe	166	13	676	0,25
Éxito	jabon bebe	104	36	147	0,71
Éxito	crema bebe	28	7	153	0,18
Éxito	aceite bebe	14	4	17	0,82
Éxito	shampoo bebe	9	5	125	0,07
Éxito	colonia bebe	5	4	243	0,02

```
cob_com <- cob |> group_by(comercio) |>
  summarise(obtenidos = sum(obtenidos), incluidos = sum(incluidos),
            con_total_sitio = sum(!is.na(total_sitio)),
            cobertura_media = round(mean(cobertura, na.rm = TRUE), 2), .groups = "drop")
tabla(cob_com, cap = "Resumen de cobertura por comercio (suma de términos; un mismo producto puede aparec
```

Tabla 9: Resumen de cobertura por comercio (suma de términos; un mismo producto puede aparecer en varios términos).

comercio	obtenidos	incluidos	con_total_sitio	cobertura_media
Carulla	104	84	6	0,47
Jumbo	104	83	5	0,80
Éxito	326	69	6	0,34

Interpretación. La columna `cobertura` solo se calcula cuando el sitio imprime el número total de resultados en el HTML; donde aparece `NA` no es posible verificar la completitud contra el sitio y la verificación se apoya en el criterio de parada (la paginación termina cuando una página no aporta productos nuevos). Los totales por término se solapan entre sí (p. ej. un jabón líquido aparece en «jabon bebe» y «shampoo bebe»), por lo que la suma por término supera la cantidad de productos únicos que muestra la sección de consultas SQL.

2.5 Qué se consideró equivalente a la categoría y qué se excluyó

Se usaron las búsquedas por término anteriores como equivalente a la categoría en cada sitio. Cada producto recibe un `motivo_exclusion` (o `NULL` si se incluye) según reglas documentadas en `motivo_exclusion()` de Scraper.R.

```
excl <- qq("SELECT comercio, COALESCE(motivo_exclusion, 'incluido') AS motivo, COUNT(*) AS n
            FROM observaciones WHERE ejecucion_id = '@EJ@'
            GROUP BY comercio, motivo ORDER BY comercio, n DESC") |>
  mutate(motivo = motivo |>
           str_replace_all("[<> →]", " ") |>
           str_replace_all("[_%&#$\\\\^~{}]", " ") |>
           str_squish()) |>
  pivot_wider(names_from = comercio, values_from = n, values_fill = 0)
tabla(excl, cap = "Productos incluidos y excluidos por motivo y comercio (ejecución analizada).",
      escape = TRUE, wrap = c(motivo = "7.5cm"))
```

Tabla 10: Productos incluidos y excluidos por motivo y comercio (ejecución analizada).

motivo	Carulla	Jumbo	Éxito
incluido	85	30	73
no es higiene personal	8	1	3
no es para bebé	6	73	327
cambio de pañal	3	0	19
kit/combo	1	0	10
fuera de subcategorías de la categoría	1	0	10
cuidado bucal	1	0	0
peinado/styling	0	0	4
protección solar	0	0	3
pomada/ungüento	0	0	2

3 Preparación de los datos

Las reglas de transformación están en `estandarizar()` (Scraper.R); aquí se resumen:

1. **Precio:** `limpiar_precio()` convierte el texto a COP numérico; el precio anterior se conserva en `precio_antes_cop` y `es_promocion = 1` cuando el precio anterior es mayor que el actual. Un precio ausente queda `NULL` (nunca 0).

2. **Marca y subcategoría:** se estandarizan con un diccionario de marcas y expresiones regulares (`MARCAS`, `SUBCATS`) en columnas propias (`marca_std`, `subcategoria_std`); el texto original (`nombre_original`, `texto_anuncio`) no se modifica.
3. **Contenido y unidad:** `extraer_contenido()` convierte L a mL y kg a g, usa el contenido **total** en paquetes (p. ej. 2×200 mL $= 400$ mL) y descarta lecturas imposibles (mayores a 5000 mL/g). `precio_100 = precio / contenido_total × 100`.
4. **Variante:** `variante_std` conserva los términos distintivos del nombre (sin marca, tamaño ni palabras genéricas).
5. **Vendedor externo y atípicos:** se marcan vendedores terceros (`vendedor_externo`) y precios por 100 mL/g fuera de 0,6×–1,6× la mediana de su grupo (`precio_atipico`). Los terceros se excluyen del emparejamiento.
6. **Duplicados:** una fila por (`comercio`, `clave_producto`, `ejecucion_id`); además, restricción `UNIQUE (comercio, clave_producto, fecha_hora_consulta)` en la base.

3.1 Del texto original al dato estandarizado

La estandarización no reemplaza el texto del anuncio: ambos conviven en la misma fila. Una muestra de los productos comparables permite ver el texto tal como se publicó junto a los campos derivados:

```
muestra <- qq("SELECT comercio, substr(nombre_original, 1, 55) AS texto_original,
                      marca_std AS marca, subcategoria_std AS subcategoria,
                      variante_std AS variante, contenido_total AS contenido, unidad,
                      precio_cop AS precio
               FROM observaciones
               WHERE comparable = 1 AND ejecucion_id = '@EJ@'
               ORDER BY producto_id, comercio LIMIT 12")
tabla(muestra,
      cap = "Muestra: texto original del anuncio y campos estandarizados derivados.",
      fs = 7, wrap = c(texto_original = "4.8cm", variante = "2.6cm"))
```

Tabla 11: Muestra: texto original del anuncio y campos estandarizados derivados.

comercio	texto_original	marca	subcategoria	variante	contenido	unidad	precio
Carulla	Baño líquido bebé JOHNSON S cabeza a pies (400 ml)	Johnson's	Jabón	cabeza pies	400	mL	29.520
Jumbo	Baño Líquido Bebé Johnson's® Baby De La Cabeza a Los Pi	Johnson's	Jabón	cabeza pies	400	mL	38.290
Éxito	Baño líquido bebé JOHNSON S cabeza a pies (400 ml)	Johnson's	Jabón	cabeza pies	400	mL	29.520
Carulla	Shampoo JOHNSON S fuerza y vitamina (400 ml)	Johnson's	Champú	fuerza vitamina	400	mL	34.400
Jumbo	Shampoo Johnson's Baby fuerza vitamina x400ml	Johnson's	Champú	fuerza vitamina	400	mL	30.950
Éxito	Shampoo JOHNSON S fuerza y vitamina (400 ml)	Johnson's	Champú	fuerza vitamina	400	mL	33.100
Carulla	Shampoo ARRURRU bebé cabello claro (750 ml)	Arrurru	Champú	cabello claro	750	mL	42.500
Jumbo	Shampoo Arrurrú cabello claro x750ml	Arrurru	Champú	cabello claro	750	mL	41.190
Éxito	Shampoo ARRURRU bebé cabello claro (750 ml)	Arrurru	Champú	cabello claro	750	mL	41.100
Carulla	Shampoo JOHNSON S gotas de brillo (400 ml)	Johnson's	Champú	brillo gotas	400	mL	21.840
Jumbo	Shampoo Johnson's Gotas de Brillo 400ml	Johnson's	Champú	brillo gotas	400	mL	29.880
Éxito	Shampoo JOHNSON S gotas de brillo (400 ml)	Johnson's	Champú	brillo gotas	400	mL	22.680

3.1.1 Conversión de unidades

Antes de calcular cualquier precio por unidad, el contenido se lleva a una unidad común. Las equivalencias aplicadas y la regla para paquetes múltiples son:

```
conv <- tibble(
  `Caso`        = c("Litros a mililitros", "Kilogramos a gramos", "Gramos a miligramos",
                    "Paquete múltiple", "Precio por unidad común"),
  `Regla`       = c("1 L = 1000 mL", "1 kg = 1000 g", "1 g = 1000 mg",
                    "contenido total = unidades × contenido de cada envase (2 × 200 mL = 400 mL)",
                    "precio_100 = precio del empaque / contenido total × 100"))
tabla(conv, cap = "Reglas de conversión de unidades usadas antes de comparar.",
      wrap = c(Regla = "9cm"))
```

Tabla 12: Reglas de conversión de unidades usadas antes de comparar.

Caso	Regla
Litros a mililitros	1 L = 1000 mL
Kilogramos a gramos	1 kg = 1000 g
Gramos a miligramos	1 g = 1000 mg
Paquete múltiple	contenido total = unidades × contenido de cada envase (2 × 200 mL = 400 mL)
Precio por unidad común	precio_100 = precio del empaque / contenido total × 100

```
unid <- q("SELECT unidad, COUNT(*) AS observaciones_comparables, COUNT(DISTINCT producto_id) AS productos
           FROM observaciones WHERE comparable = 1 GROUP BY unidad ORDER BY unidad")
tabla(unid, cap = "Unidades presentes en los productos comparables (nunca se mezclan en un mismo ranking)")
```

Tabla 13: Unidades presentes en los productos comparables (nunca se mezclan en un mismo ranking).

unidad	observaciones_comparables	productos
g	6	1
mL	66	11

3.2 Correspondencia de productos entre comercios

`proponer_por_niveles()` empareja productos por marca, tipo y unidad, y asigna un nivel de confianza (A: misma presentación; B: misma línea, distinto tamaño; C: nombre poco parecido; D: familia de producto). El producto común se registra en la base con `producto_id` y `comparable = 1` mediante `fijar_comparables_linea()`.

```
datos_ult <- cargar_datos(DB)
cand <- tryCatch(proponer_por_niveles(datos_ult), error = function(e) tibble())
if (nrow(cand) > 0) {
  tb <- armar_tablas(cand)
  tabla(count(tb$tabla, nivel, name = "candidatos"),
        cap = "Candidatos a comparables detectados por nivel de confianza.")
} else {
  cat("No se generaron candidatos con los datos actuales.\n")
}
```

Tabla 14: Candidatos a comparables detectados por nivel de confianza.

nivel	candidatos
A. Misma presentación	18
B. Misma línea, distinto tamaño	1
C. Misma línea, nombre poco parecido (revisar)	4
D. Misma marca y tipo, otra variante (familia)	6

```
corr <- qq("SELECT p.producto_id AS id, p.nombre_comun AS producto, o.comercio,
                   o.contenido_total AS contenido, o.unidad, o.precio_cop AS precio,
                   substr(o.nombre_original, 1, 60) AS nombre_en_el_sitio
            FROM observaciones o JOIN productos p ON p.producto_id = o.producto_id
            WHERE o.comparable = 1 AND o.ejecucion_id = '@EJ@'
            ORDER BY p.producto_id, o.comercio")

# nivel de confianza de cada producto registrado (vía clave del producto en Éxito)
if (nrow(cand) > 0) {
  claves <- q("SELECT DISTINCT p.producto_id AS id, o.clave_producto AS clave
               FROM observaciones o JOIN productos p ON p.producto_id = o.producto_id
               WHERE o.comercio = 'Éxito' AND o.comparable = 1")
  niv <- cand |> transmute(clave = as.character(clave_producto_Exito), nivel) |>
    inner_join(claves, by = "clave") |> select(id, nivel)
  corr <- left_join(corr, niv, by = "id")
}
tabla(corr, cap = "Correspondencia registrada en la base (misma ejecución).",
      fs = 7, wrap = c(producto = "3.2cm", nombre_en_el_sitio = "4.6cm"))
```

Tabla 15: Correspondencia registrada en la base (misma ejecución).

id	producto	comercio	contenido	unidad	precio	nombre_en_el_sitio	nivel
P01	Johnson's Jabón cabeza pies	Carulla	400	mL	29.520	Baño líquido bebé JOHNSON S cabeza a pies (400 ml)	D. Misma marca y tipo, otra variante (familia
P01	Johnson's Jabón cabeza pies	Jumbo	400	mL	38.290	Baño Líquido Bebé Johnson's® Baby De La Cabeza a Los Pies 40	D. Misma marca y tipo, otra variante (familia
P01	Johnson's Jabón cabeza pies	Éxito	400	mL	29.520	Baño líquido bebé JOHNSON S cabeza a pies (400 ml)	D. Misma marca y tipo, otra variante (familia
P02	Johnson's Champú fuerza vitamina	Carulla	400	mL	34.400	Shampoo JOHNSON S fuerza y vitamina (400 ml)	
P02	Johnson's Champú fuerza vitamina	Jumbo	400	mL	30.950	Shampoo Johnson's Baby fuerza vitamina x400ml	
P02	Johnson's Champú fuerza vitamina	Éxito	400	mL	33.100	Shampoo JOHNSON S fuerza y vitamina (400 ml)	
P03	Arrurru Champú cabello claro	Carulla	750	mL	42.500	Shampoo ARRURRU bebé cabello claro (750 ml)	A. Misma presentación
P03	Arrurru Champú cabello claro	Jumbo	750	mL	41.190	Shampoo Arrurrú cabello claro x750ml	A. Misma presentación
P03	Arrurru Champú cabello claro	Éxito	750	mL	41.100	Shampoo ARRURRU bebé cabello claro (750 ml)	A. Misma presentación
P04	Johnson's Champú brillo gotas	Carulla	400	mL	21.840	Shampoo JOHNSON S gotas de brillo (400 ml)	A. Misma presentación
P04	Johnson's Champú brillo gotas	Jumbo	400	mL	29.880	Shampoo Johnson's Gotas de Brillo 400ml	A. Misma presentación
P04	Johnson's Champú brillo gotas	Éxito	400	mL	22.680	Shampoo JOHNSON S gotas de brillo (400 ml)	A. Misma presentación
P05	Johnson's Champú hidratacion intensa	Carulla	400	mL	27.300	Shampoo JOHNSON S bebé hidratación intensa (400 ml)	A. Misma presentación
P05	Johnson's Champú hidratacion intensa	Jumbo	400	mL	29.430	Shampoo Johnson'S Hidratación Intensa 400 ml	A. Misma presentación
P05	Johnson's Champú hidratacion intensa	Éxito	400	mL	25.950	Shampoo JOHNSON S bebé hidratación intensa (400 ml)	A. Misma presentación

Tabla 15: Correspondencia registrada en la base (misma ejecución). *(continued)*

id	producto	comercio	contenido	unidad	precio	nombre_en_el_sitio	nivel
P06	Johnson's Champú original	Carulla	400	mL	24.800	Shampoo JOHNSON S baby original (400 ml)	A. Misma presentación
P06	Johnson's Champú original	Jumbo	400	mL	32.220	Shampoo Bebé Johnson'S Original 400 ml	A. Misma presentación
P06	Johnson's Champú original	Éxito	750	mL	32.160	Shampoo JOHNSON S baby original (750 ml)	A. Misma presentación
P07	Arrurru Colonia arruru azul original	Carulla	220	mL	36.700	Colonia para bebé ARRURRU original azul (220 ml)	A. Misma presentación
P07	Arrurru Colonia arruru azul original	Jumbo	120	mL	27.790	Colonia Arrurru original azul x120ml + Jabón Arruru x110g	A. Misma presentación
P07	Arrurru Colonia arruru azul original	Éxito	220	mL	36.300	Colonia para bebé ARRURRU original azul (220 ml)	A. Misma presentación
P08	Johnson's Champú original	Carulla	200	mL	19.700	Shampoo JOHNSON S baby original (200 ml)	C. Misma línea, nombre poco parecido (revisa
P08	Johnson's Champú original	Jumbo	200	mL	21.640	Shampoo Bebé Johnson'S Original 200 ml	C. Misma línea, nombre poco parecido (revisa
P08	Johnson's Champú original	Éxito	750	mL	32.160	Shampoo JOHNSON S baby original (750 ml)	C. Misma línea, nombre poco parecido (revisa
P09	Johnson's Jabón avena tripack	Carulla	660	g	18.240	Jabón de baño JOHNSON S avena (660 gr)	
P09	Johnson's Jabón avena tripack	Jumbo	108.900	g	16.020	Jabón Bebé Johnson'S Avena Tripack 330 gr	
P09	Johnson's Jabón avena tripack	Éxito	660	g	18.240	Jabón de baño JOHNSON S avena (660 gr)	
P10	Arrurru Colonia arruru original rosa	Carulla	220	mL	35.500	Colonia para bebé ARRURRU original rosada (220 ml)	A. Misma presentación
P10	Arrurru Colonia arruru original rosa	Jumbo	120	mL	27.790	Colonia Arrurru original rosa x120ml + Jabón Arruru x110g	A. Misma presentación
P10	Arrurru Colonia arruru original rosa	Éxito	220	mL	35.500	Colonia para bebé ARRURRU original rosada (220 ml)	A. Misma presentación
P11	Johnson's Champú cabello claro manzanilla	Carulla	400	mL	21.840	Shampoo bebé JOHNSON S baby manzanilla cabello claro (400 ml	D. Misma marca y tipo, otra variante (familia
P11	Johnson's Champú cabello claro manzanilla	Jumbo	400	mL	30.950	Shampoo Bebé Johnson's® Baby Manzanilla Cabello Claro 400 ml	D. Misma marca y tipo, otra variante (familia
P11	Johnson's Champú cabello claro manzanilla	Éxito	400	mL	24.560	Shampoo JOHNSON S Aco 2 EN 1 Kids (400 ml)	D. Misma marca y tipo, otra variante (familia
P12	Baby Soft Champú docil nudos	Carulla	750	mL	31.400	Shampoo BABY SOFT Cabello Claro Manzanilla (750 ml)	D. Misma marca y tipo, otra variante (familia
P12	Baby Soft Champú docil nudos	Jumbo	750	mL	32.990	Shampoo Baby Soft Dócil y Sin Nudos 750 ml	D. Misma marca y tipo, otra variante (familia
P12	Baby Soft Champú docil nudos	Éxito	750	mL	31.050	Shampoo BABY SOFT Vainilla y Proteina de Soya (750 ml)	D. Misma marca y tipo, otra variante (familia

Interpretación. Cada `id` agrupa un artículo por comercio. Donde el contenido difiere entre comercios (niveles B–D) las comparaciones se hacen **por precio por 100 mL/g** y no por precio del empaque.

3.2.1 Verificación de la correspondencia

Para que dos filas de comercios distintos se consideren el mismo producto deben coincidir la **marca**, la **variante**, el **contenido** (incluyendo el número de unidades en paquetes) y la **unidad de medida**. Como los productos de esta categoría no declaran concentración, esa condición no aplica. Cada producto registrado se revisa con estos criterios:

```
ver <- qq("SELECT p.producto_id AS id, substr(p.nombre_comun, 1, 38) AS producto,
                  COUNT(DISTINCT o.comercio)      AS comercios,
                  COUNT(DISTINCT o.marca_std)     AS marcas,
                  COUNT(DISTINCT o.variante_std)  AS variantes,
                  COUNT(DISTINCT o.unidad)        AS unidades,
                  ROUND(MAX(o.contenido_total) * 1.0 / MIN(o.contenido_total), 2) AS razon_contenido
```

```
          FROM observaciones o JOIN productos p ON p.producto_id = o.producto_id
          WHERE o.comparable = 1 AND o.ejecucion_id = '@EJ@'
          GROUP BY p.producto_id, p.nombre_comun ORDER BY p.producto_id") |>
  mutate(resultado = case_when(
    comercios < 3 | marcas > 1 | unidades > 1 ~ "revisar",
    variantes > 1                             ~ "revisar variante",
    razon_contenido > 1.03                    ~ "tamaño distinto",
    TRUE                                      ~ "idéntica"))
tabla(ver, cap = "Verificación de la correspondencia: marca, variante, unidad y razón de contenido (máx./
      fs = 7.5, wrap = c(producto = "4.8cm"))
```

Tabla 16: Verificación de la correspondencia: marca, variante, unidad y razón de contenido (máx./mín.) por producto.

id	producto	comercios	marcas	variantes	unidades	razon_contenido	resultado
P01	Johnson's Jabón cabeza pies	3	1	1	1	1,00	idéntica
P02	Johnson's Champú fuerza vitamina	3	1	1	1	1,00	idéntica
P03	Arrurru Champú cabello claro	3	1	1	1	1,00	idéntica
P04	Johnson's Champú brillo gotas	3	1	1	1	1,00	idéntica
P05	Johnson's Champú hidratacion intensa	3	1	1	1	1,00	idéntica
P06	Johnson's Champú original	3	1	1	1	1,88	tamaño distinto
P07	Arrurru Colonia arruru azul original	3	1	2	1	1,83	revisar variante
P08	Johnson's Champú original	3	1	1	1	3,75	tamaño distinto
P09	Johnson's Jabón avena tripack	3	1	2	1	165,00	revisar variante
P10	Arrurru Colonia arruru original rosa	3	1	2	1	1,83	revisar variante
P11	Johnson's Champú cabello claro manzani	3	1	2	1	1,00	revisar variante
P12	Baby Soft Champú docil nudos	3	2	3	1	1,00	revisar

Interpretación. Hay 5 productos con presentación idéntica en los tres comercios (misma marca, variante, unidad y contenido dentro de ±3 %), 2 con la misma línea pero tamaño distinto, y 5 que requieren revisión manual. Solo las presentaciones idénticas se comparan por precio del empaque; las demás se comparan por precio por 100 mL/g. Un producto presente en un solo comercio no cuenta como comparable.

4 Base SQLite

```
tablas  <- dbListTables(con)
esquema <- map_dfr(tablas, function(tb_nombre) {
  tibble(tabla = tb_nombre,
         columnas = paste(dbListFields(con, tb_nombre), collapse = ", "))
})
tabla(esquema, cap = "Tablas de la base de datos SQLite del comparador.",
      wrap = c(columnas = "11cm"))
```

Tabla 17: Tablas de la base de datos SQLite del comparador.

tabla	columnas
ejecuciones	ejecucion_id, comercio, fecha_hora_inicio, n_registros, estado, modo

observaciones	obs_id, ejecucion_id, comercio, clave_producto, sku, url_producto, url_busqueda, termino_busqueda, nombre_original, texto_anuncio, marca_std, subcategoria_std, precio_cop, precio_antes_cop, es_promocion, contenido_unitario, unidad, unidades_pack, contenido_total, incluido, motivo_exclusion, producto_id, comparable, fecha_hora_consulta, publico, marca_sitio, precio_tarjeta_cop, precio_100, variante_std, clave_relajada, vendedor_externo, precio_atipico
productos	producto_id, nombre_comun, marca, subcategoria, contenido, unidad, unidades_pack
sqlite_sequence	name, seq

- `ejecuciones`: una fila por corrida y comercio (identificador, fecha/hora de inicio en ISO 8601, número de registros, estado `completa/parcial/fallida`, modo).
- `productos`: identificador común (`producto_id`) de los productos comparables.
- `observaciones`: un precio por producto, comercio y momento de consulta; conserva el texto original, URL, SKU, precio anterior, promoción, contenido, `precio_100`, `comparable` y `ejecucion_id`. Unicidad: `(comercio, clave_producto, fecha_hora_consulta)`.

4.1 Estructura de la base

La base separa tres cosas: lo que se **ejecutó** (`ejecuciones`), el **producto común** que permite comparar entre comercios (`productos`) y cada **precio observado** en un comercio y momento dado (`observaciones`). Las relaciones lógicas son:

```
ejecuciones                          productos
(ejecucion_id, comercio, ...)        (producto_id, nombre_comun, ...)
        1                                    1


                      N       N
                  observaciones
        (comercio, clave_producto, fecha_hora_consulta  ← UNIQUE
         ejecucion_id, producto_id, precio_cop, precio_100, comparable, ...)
```

Definición de columnas, tipos y llaves de cada tabla:

```
tablas_usuario <- setdiff(dbListTables(con), grep("^sqlite_", dbListTables(con), value = TRUE))
estr <- map_dfr(tablas_usuario, function(tb_nombre) {
  as_tibble(dbGetQuery(con, sprintf("PRAGMA table_info('%s')", tb_nombre))) |>
    transmute(tabla = tb_nombre, columna = name, tipo = ifelse(type == "", "-", type),
              obligatorio = ifelse(notnull == 1, "sí", ""),
              llave_primaria = ifelse(pk > 0, "sí", ""))
})
tabla(estr, cap = "Estructura de las tablas de la base: columnas, tipo, obligatoriedad y llave primaria."
      fs = 7.5)
```

Tabla 18: Estructura de las tablas de la base: columnas, tipo, obligatoriedad y llave primaria.

tabla	columna	tipo	obligatorio	llave_primaria
ejecuciones	ejecucion_id	TEXT	sí	sí
ejecuciones	comercio	TEXT	sí	sí
ejecuciones	fecha_hora_inicio	TEXT	sí	
ejecuciones	n_registros	INTEGER		
ejecuciones	estado	TEXT		
ejecuciones	modo	TEXT		
observaciones	obs_id	INTEGER		sí

Tabla 18: Estructura de las tablas de la base: columnas, tipo, obligatoriedad y llave primaria. *(continued)*

tabla	columna	tipo	obligatorio	llave_primaria
observaciones	ejecucion_id	TEXT	sí	
observaciones	comercio	TEXT	sí	
observaciones	clave_producto	TEXT	sí	
observaciones	sku	TEXT		
observaciones	url_producto	TEXT		
observaciones	url_busqueda	TEXT		
observaciones	termino_busqueda	TEXT		
observaciones	nombre_original	TEXT		
observaciones	texto_anuncio	TEXT		
observaciones	marca_std	TEXT		
observaciones	subcategoria_std	TEXT		
observaciones	precio_cop	REAL		
observaciones	precio_antes_cop	REAL		
observaciones	es_promocion	INTEGER		
observaciones	contenido_unitario	REAL		
observaciones	unidad	TEXT		
observaciones	unidades_pack	REAL		
observaciones	contenido_total	REAL		
observaciones	incluido	INTEGER		
observaciones	motivo_exclusion	TEXT		
observaciones	producto_id	TEXT		
observaciones	comparable	INTEGER		
observaciones	fecha_hora_consulta	TEXT	sí	
observaciones	publico	TEXT		
observaciones	marca_sitio	TEXT		
observaciones	precio_tarjeta_cop	REAL		
observaciones	precio_100	REAL		
observaciones	variante_std	TEXT		
observaciones	clave_relajada	TEXT		
observaciones	vendedor_externo	INTEGER		
observaciones	precio_atipico	INTEGER		
productos	producto_id	TEXT		sí
productos	nombre_comun	TEXT		
productos	marca	TEXT		
productos	subcategoria	TEXT		
productos	contenido	REAL		
productos	unidad	TEXT		
productos	unidades_pack	REAL		

```
# Índices y restricciones de unicidad
idx_tb <- map_dfr(tablas_usuario, function(tb_nombre) {
  ix <- as_tibble(dbGetQuery(con, sprintf("PRAGMA index_list('%s')", tb_nombre)))
  if (nrow(ix) == 0) return(tibble())
  map_dfr(seq_len(nrow(ix)), function(i) {
    cols <- dbGetQuery(con, sprintf("PRAGMA index_info('%s')", ix$name[i]))$name
    tibble(tabla = tb_nombre, indice = ix$name[i],
           unico = ifelse(ix$unique[i] == 1, "sí", "no"),
           columnas = paste(cols, collapse = ", "))
  })
})
if (nrow(idx_tb) > 0) {
  tabla(idx_tb, cap = "Índices y restricciones de unicidad definidos en la base.",
        fs = 7.5, wrap = c(indice = "4.5cm", columnas = "6cm"))
}
```

Tabla 19: Índices y restricciones de unicidad definidos en la base.

tabla	indice	unico	columnas
ejecuciones	sqlite_autoindex_ejecuciones_1	sí	ejecucion_id, comercio
observaciones	sqlite_autoindex_observaciones_1	sí	comercio, clave_producto, fecha_hora_consulta
productos	sqlite_autoindex_productos_1	sí	producto_id

```
# Llaves foráneas (si están declaradas en el esquema)
fk <- map_dfr(tablas_usuario, function(tb_nombre) {
  d <- dbGetQuery(con, sprintf("PRAGMA foreign_key_list('%s')", tb_nombre))
  if (nrow(d) == 0) return(tibble())
  tibble(tabla = tb_nombre, from = as.character(d$from),
         table = as.character(d$table), to = as.character(d$to))
})
if (nrow(fk) > 0) {
  tabla(fk |> transmute(tabla, columna = from, referencia = paste0(table, "(", to, ")")),
        cap = "Llaves foráneas declaradas.")
}
```

Tabla 20: Llaves foráneas declaradas.

tabla	columna	referencia
observaciones	producto_id	productos(producto_id)
observaciones	ejecucion_id	ejecuciones(ejecucion_id)

```
conteos <- map_dfr(tablas_usuario, function(tb_nombre) {
  tibble(tabla = tb_nombre,
         filas = as.integer(dbGetQuery(con, sprintf("SELECT COUNT(*) AS n FROM \"%s\"", tb_nombre))$n),
         columnas = length(dbListFields(con, tb_nombre)))
})
tabla(conteos, cap = "Tamaño actual de cada tabla de la base.")
```

Tabla 21: Tamaño actual de cada tabla de la base.

tabla	filas	columnas
ejecuciones	9	6
observaciones	1.811	32
productos	12	7

4.1.1 Campos mínimos y tabla donde se guardan

```
req <- tribble(
  ~requisito,                    ~patron,
  "Comercio",                    "^comercio$",
  "Producto común",              "^producto_id$",
  "Marca estandarizada",         "^marca_std$",
  "Subcategoría estandarizada",  "^subcategoria_std$",
  "Presentación (contenido)",    "^contenido_total$|^unidad$",
  "Precio (COP)",                "^precio_cop$",
  "Promoción",                   "^es_promocion$",
  "URL de extracción",           "url",
```

```
  "Texto original del anuncio",   "^nombre_original$|^texto_anuncio$",
  "Fecha y hora de consulta",     "^fecha_hora_consulta$",
  "Comparable",                   "^comparable$",
  "Ejecución",                    "^ejecucion_id$")
donde <- function(patron) {
  map_dfr(tablas_usuario, function(tb_nombre) {
    g <- grep(patron, dbListFields(con, tb_nombre), value = TRUE)
    if (length(g)) tibble(tabla = tb_nombre, columna = paste(g, collapse = ", ")) else tibble()
  })
}
req_tb <- req |>
  mutate(ubicacion = map_chr(patron, function(p) {
    d <- donde(p)
    if (nrow(d) == 0) "(no encontrada)" else
      paste0(d$tabla, ".", d$columna, collapse = "; ") })) |>
  select(requisito, ubicacion)
tabla(req_tb, cap = "Información mínima requerida y columna donde se guarda en la base.",
      wrap = c(requisito = "4.8cm", ubicacion = "9cm"))
```

Tabla 22: Información mínima requerida y columna donde se guarda en la base.

requisito	ubicacion
Comercio	ejecuciones.comercio; observaciones.comercio
Producto común	observaciones.producto_id; productos.producto_id
Marca estandarizada	observaciones.marca_std
Subcategoría estandarizada	observaciones.subcategoria_std
Presentación (contenido)	observaciones.unidad, contenido_total; productos.unidad
Precio (COP)	observaciones.precio_cop
Promoción	observaciones.es_promocion
URL de extracción	observaciones.url_producto, url_busqueda
Texto original del anuncio	observaciones.nombre_original, texto_anuncio
Fecha y hora de consulta	observaciones.fecha_hora_consulta
Comparable	observaciones.comparable
Ejecución	ejecuciones.ejecucion_id; observaciones.ejecucion_id

Interpretación. El producto común (`producto_id`) vive en `productos` y se enlaza con cada observación, de modo que la misma presentación se reconoce sin ambigüedad entre comercios. La tabla `observaciones` conserva cada precio junto a su fecha y hora de consulta y a su ejecución, por lo que las corridas nuevas se agregan sin sobrescribir las anteriores y se puede comparar el valor actual con observaciones previas del mismo producto y comercio. La restricción `UNIQUE (comercio, clave_producto, fecha_hora_consulta)` impide insertar dos veces la misma observación.

```
tabla(q("SELECT * FROM ejecuciones ORDER BY ejecucion_id DESC, comercio"),
      cap = "Registro de ejecuciones (cada corrida conserva sus observaciones; no se sobrescriben).",
      fs = 7)
```

Tabla 23: Registro de ejecuciones (cada corrida conserva sus observaciones; no se sobrescriben).

ejecucion_id	comercio	fecha_hora_inicio	n_registros	estado	modo
2026-10-08T225121	Carulla	2026-10-09T08:35:49-05:00	104	completa	offline
2026-10-08T225121	Jumbo	2026-10-09T08:35:55-05:00	104	completa	offline
2026-10-08T225121	Éxito	2026-10-09T08:35:26-05:00	326	completa	offline
2026-10-07T110040	Carulla	2026-10-07T19:11:45-05:00	105	completa	offline
2026-10-07T110040	Jumbo	2026-10-07T19:11:52-05:00	104	completa	offline

2026-10-07T110040	Éxito	2026-10-07T19:11:14-05:00	451	completa	offline
2026-10-07T101436	Carulla	2026-10-07T10:28:27-05:00	104	completa	online
2026-10-07T101436	Jumbo	2026-10-07T10:33:03-05:00	62	completa	online
2026-10-07T101436	Éxito	2026-10-07T10:14:36-05:00	451	completa	online

4.2 Validaciones

```
# 1) Restricción de unicidad definida en el esquema
unica <- q("SELECT name AS tabla, sql FROM sqlite_master WHERE type = 'table'") |>
  transmute(tabla, tiene_UNIQUE = ifelse(grepl("UNIQUE", sql, ignore.case = TRUE), "sí", "no")) |>
  filter(!grepl("^sqlite_", tabla))
tabla(unica, cap = "Validación: tablas con restricción UNIQUE en su definición.")
```

Tabla 24: Validación: tablas con restricción UNIQUE en su definición.

tabla	tiene_UNIQUE
ejecuciones	no
productos	no
observaciones	sí

```
# 2) Calidad de los campos obligatorios en observaciones
val <- q("
SELECT COUNT(*)                                              AS observaciones,
       SUM(fecha_hora_consulta IS NULL)                      AS sin_fecha_hora,
       SUM(fecha_hora_consulta NOT LIKE '____-__-__T__:__:__-05:00') AS fecha_no_iso,
       SUM(precio_cop = 0)                                   AS precio_cero,
       SUM(precio_cop IS NULL)                               AS precio_nulo
FROM observaciones")
tabla(val, cap = "Validación: fecha y hora de consulta en ISO 8601 (-05:00) y precios (los ausentes queda
```

Tabla 25: Validación: fecha y hora de consulta en ISO 8601 (-05:00) y precios (los ausentes quedan NULL, nunca 0).

observaciones	sin_fecha_hora	fecha_no_iso	precio_cero	precio_nulo
1.811	0	1.194	1	62

```
# 3) Observaciones repetidas según la clave de unicidad
dup <- q("SELECT COUNT(*) AS n FROM (
            SELECT 1 FROM observaciones
            GROUP BY comercio, clave_producto, fecha_hora_consulta HAVING COUNT(*) > 1)")$n
```

Interpretación. Se espera `sin_fecha_hora = 0`, `fecha_no_iso = 0` y `precio_cero = 0`. Observaciones repetidas según la clave (comercio, producto del comercio, fecha y hora): **0**.

5 Consultas SQL

Las consultas de comparables usan la ejecución 2026-10-07T110040. Para los precios del empaque (preguntas 3, 4 y parte de la 6) solo se consideran presentaciones **equivalentes** (diferencia de contenido de máximo 3 %); el resto se compara por precio por 100 mL/g.

5.1 Pregunta 1 – Tamaño de la base

```
sql1a <- "
SELECT COUNT(DISTINCT comercio)                          AS comercios,
       COUNT(DISTINCT comercio || '|' || clave_producto) AS productos_por_comercio,
       COUNT(DISTINCT producto_id)                       AS productos_comparables,
       COUNT(DISTINCT ejecucion_id)                      AS ejecuciones,
       COUNT(*)                                          AS observaciones
FROM observaciones"
sql1b <- "
SELECT ejecucion_id, comercio,
       COUNT(DISTINCT clave_producto) AS productos,
       SUM(incluido = 1)              AS incluidos,
       COUNT(*)                       AS observaciones
FROM observaciones GROUP BY ejecucion_id, comercio ORDER BY ejecucion_id DESC, comercio"
q1a <- q(sql1a); q1b <- q(sql1b)
tabla(q1a, cap = "Pregunta 1: totales de la base.")
```

Tabla 26: Pregunta 1: totales de la base.

comercios	productos_por_comercio	productos_comparables	ejecuciones	observaciones
3	770	12	3	1.811

```
tabla(q1b, cap = "Pregunta 1: productos aportados por cada comercio y ejecución.")
```

Tabla 27: Pregunta 1: productos aportados por cada comercio y ejecución.

ejecucion_id	comercio	productos	incluidos	observaciones
2026-10-08T225121	Carulla	104	84	104
2026-10-08T225121	Jumbo	104	83	104
2026-10-08T225121	Éxito	326	69	326
2026-10-07T110040	Carulla	105	85	105
2026-10-07T110040	Jumbo	104	30	104
2026-10-07T110040	Éxito	451	73	451
2026-10-07T101436	Carulla	104	78	104
2026-10-07T101436	Jumbo	62	39	62
2026-10-07T101436	Éxito	451	73	451

Interpretación. La base contiene 3 comercios, 770 productos distintos (por comercio), 12 productos comparables y 1.811 observaciones en 3 ejecución(es). La tabla inferior muestra cuántos productos aporta cada comercio y cuántos pasan los filtros de la categoría (`incluidos`).

5.2 Pregunta 2 – Cobertura obligatoria y duplicados

```
sql2a <- "
SELECT ejecucion_id, COUNT(*) AS obs_comparables,
       COUNT(DISTINCT producto_id) AS productos, COUNT(DISTINCT comercio) AS comercios,
       CASE WHEN COUNT(*) >= 24 AND COUNT(DISTINCT producto_id) >= 8
                 AND COUNT(DISTINCT comercio) = 3 THEN 'sí' ELSE 'no' END AS cumple_24
FROM observaciones WHERE comparable = 1 AND precio_cop IS NOT NULL
```

```
GROUP BY ejecucion_id ORDER BY ejecucion_id DESC"
sql2b <- "
SELECT ejecucion_id, producto_id, comercio, COUNT(*) AS veces
FROM observaciones WHERE comparable = 1
GROUP BY ejecucion_id, producto_id, comercio HAVING COUNT(*) > 1"
q2a <- q(sql2a); q2b <- q(sql2b)
tabla(q2a, cap = "Pregunta 2: observaciones comparables con precio por ejecución.")
```

Tabla 28: Pregunta 2: observaciones comparables con precio por ejecución.

ejecucion_id	obs_comparables	productos	comercios	cumple_24
2026-10-07T110040	36	12	3	sí
2026-10-07T101436	24	12	2	no

```
if (nrow(q2b) == 0) {
  cat("No hay registros repetidos del mismo producto y comercio en una ejecución.\n")
} else {
  tabla(q2b, cap = "Registros repetidos (producto, comercio, ejecución).")
}
```

```
No hay registros repetidos del mismo producto y comercio en una ejecución.
```

```
fila <- q2a |> filter(ejecucion_id == EJ)
cumple <- nrow(fila) == 1 && fila$cumple_24 == "sí"
```

Interpretación. En la ejecución 2026-10-07T110040 se obtuvieron 36 observaciones comparables con precio, de 12 productos en 3 comercios: **se cumple** el mínimo de 24 (8×3). No hay duplicados, así que cada producto aporta una sola observación por comercio.

5.3 Pregunta 3 – Precio mínimo, máximo y promedio (misma presentación)

```
sql3 <- paste0(SQL_BASE, "
SELECT t.producto_id AS id, t.nombre_comun AS producto, t.cont_min AS contenido, t.unidad,
       t.n_comercios AS n_com, t.p_min AS minimo, t.p_prom AS promedio, t.p_max AS maximo,
       (SELECT group_concat(c.comercio, ' = ') FROM c
         WHERE c.producto_id = t.producto_id AND c.precio_cop = t.p_min) AS comercio_mas_barato
FROM t WHERE t.cont_max * 1.0 / t.cont_min <= 1.03
ORDER BY t.producto_id")
q3 <- qq(sql3)
tabla(q3 |> rename(`comercio más barato` = comercio_mas_barato),
      cap = "Pregunta 3: precio del empaque (COP) en presentaciones equivalentes; los empates se muestran
      fs = 7.5, wrap = c(producto = "3.8cm", `comercio más barato` = "2.4cm"))
```

Tabla 29: Pregunta 3: precio del empaque (COP) en presentaciones equivalentes; los empates se muestran con «=».

id	producto	contenido	unidad	n_com	minimo	promedio	maximo	comercio más barato
P01	Johnson's Jabón cabeza pies	400	mL	3	29.520	32.443	38.290	Éxito = Carulla

P02	Johnson's Champú fuerza vitamina	400	mL	3	30.950	32.817	34.400	Jumbo
P03	Arrurru Champú cabello claro	750	mL	3	41.100	41.597	42.500	Éxito
P04	Johnson's Champú brillo gotas	400	mL	3	21.840	24.800	29.880	Carulla
P05	Johnson's Champú hidratacion intensa	400	mL	3	25.950	27.560	29.430	Éxito
P11	Johnson's Champú cabello claro manzanilla	400	mL	3	21.840	25.783	30.950	Carulla
P12	Baby Soft Champú docil nudos	750	mL	3	31.050	31.813	32.990	Éxito

Interpretación. 7 productos tienen exactamente la misma presentación en los comercios. El comercio más barato por empaque es Éxito en el mayor número de ellos (contando empates). Un promedio entre tres precios sirve solo como referencia; la decisión de compra se apoya en el mínimo.

5.4 Pregunta 4 – Ahorro al elegir la oferta más económica

```
sql4 <- paste0(SQL_BASE, "
SELECT t.producto_id AS id, t.nombre_comun AS producto, t.p_min AS pmin, t.p_max AS pmax,
       t.p_max - t.p_min AS ahorro_cop,
       ROUND(100.0 * (t.p_max - t.p_min) / t.p_max, 1) AS ahorro_pct
FROM t WHERE t.cont_max * 1.0 / t.cont_min <= 1.03 AND t.n_comercios = 3
ORDER BY ahorro_pct DESC")
q4 <- qq(sql4)
tabla(q4, cap = "Pregunta 4: ahorro en COP = Pmax - Pmin y ahorro en porcentaje = 100 (Pmax - Pmin) / Pma
      wrap = c(producto = "5.5cm"))
```

Tabla 30: Pregunta 4: ahorro en COP = Pmax - Pmin y ahorro en porcentaje = 100 (Pmax - Pmin) / Pmax.

id	producto	pmin	pmax	ahorro_cop	ahorro_pct
P11	Johnson's Champú cabello claro manzanilla	21.840	30.950	9.110	29,4
P04	Johnson's Champú brillo gotas	21.840	29.880	8.040	26,9
P01	Johnson's Jabón cabeza pies	29.520	38.290	8.770	22,9
P05	Johnson's Champú hidratacion intensa	25.950	29.430	3.480	11,8
P02	Johnson's Champú fuerza vitamina	30.950	34.400	3.450	10,0
P12	Baby Soft Champú docil nudos	31.050	32.990	1.940	5,9
P03	Arrurru Champú cabello claro	41.100	42.500	1.400	3,3

Interpretación. El ahorro medio al comprar en el comercio más barato es de **5.170 COP** (15.7 %) por producto, con un máximo de 9.110 COP. Es un ahorro por unidad comprada, no por canasta.

5.4.1 Por qué no se comparan promedios de surtidos distintos

El ahorro solo tiene sentido para **una misma presentación y condiciones comparables**. Promediar todo lo que vende cada comercio mezcla surtidos diferentes (distintas marcas, tamaños y subcategorías) y no mide qué comercio es más barato:

```
prom <- qq("SELECT comercio, unidad, COUNT(*) AS productos,
                   ROUND(AVG(precio_100), 1) AS promedio_x100
            FROM observaciones
            WHERE ejecucion_id = '@EJ@' AND incluido = 1 AND precio_100 IS NOT NULL
            GROUP BY comercio, unidad ORDER BY unidad, comercio")
tabla(prom, cap = "Promedio de COP por 100 mL/g de todo el surtido incluido: NO es una comparación válida
```

Tabla 31: Promedio de COP por 100 mL/g de todo el surtido incluido: NO es una comparación válida entre comercios (los surtidos difieren).

comercio	unidad	productos	promedio_x100
Carulla	g	9	24.777,7
Jumbo	g	9	1.808,7
Éxito	g	5	30.814,1
Carulla	mL	73	87.522,7
Jumbo	mL	21	10.549,8
Éxito	mL	61	9.212,2

```
sql_can <- paste0(SQL_BASE, "
SELECT c.comercio, COUNT(*) AS productos, SUM(c.precio_cop) AS total
FROM c JOIN t ON t.producto_id = c.producto_id
WHERE t.cont_max * 1.0 / t.cont_min <= 1.03 AND t.n_comercios = 3
GROUP BY c.comercio ORDER BY total")
can_sql <- qq(sql_can) |>
  mutate(`diferencia vs. más barato` = total - min(total))
tabla(can_sql, cap = "Comparación válida: canasta con exactamente las mismas presentaciones y cantidades
```

Tabla 32: Comparación válida: canasta con exactamente las mismas presentaciones y cantidades (una unidad de cada una), misma ejecución y cobertura completa.

comercio	productos	total	diferencia vs. más barato
Éxito	7	207.960	0
Carulla	7	208.800	840
Jumbo	7	233.680	25.720

Interpretación. Los promedios del primer cuadro cambian según qué productos tenga cada comercio, por lo que no permiten concluir que uno sea «globalmente» más barato. La canasta del segundo cuadro sí es comparable: contiene las mismas 7 presentaciones en los tres comercios. Para esa canasta, el ahorro entre la más cara y la más barata es de 25.720 COP (11 %), calculado con Ahorro (COP) = Pmax − Pmin y Ahorro (%) = 100 (Pmax − Pmin) / Pmax.

5.5 Pregunta 5 – Menor precio por 100 mL / 100 g

```
sql5 <- paste0(SQL_BASE, "
SELECT producto_id AS id, nombre_comun AS producto, comercio,
       contenido_total AS contenido, unidad, precio_cop AS precio,
       ROUND(precio_100, 1) AS precio_x100,
       RANK() OVER (PARTITION BY producto_id ORDER BY ROUND(precio_100, 2)) AS puesto
FROM c ORDER BY producto_id, puesto, comercio")
q5 <- qq(sql5)
tabla(q5, cap = "Pregunta 5: precio por 100 mL o 100 g (no se mezclan unidades en un mismo ranking); pues
      fs = 7.5, wrap = c(producto = "5cm"))
```

Tabla 33: Pregunta 5: precio por 100 mL o 100 g (no se mezclan unidades en un mismo ranking); puesto 1 = más barato.

id	producto	comercio	contenido	unidad	precio	precio_x100	puesto
P01	Johnson's Jabón cabeza pies	Carulla	400	mL	29.520	7.380,0	1
P01	Johnson's Jabón cabeza pies	Éxito	400	mL	29.520	7.380,0	1
P01	Johnson's Jabón cabeza pies	Jumbo	400	mL	38.290	9.572,5	3
P02	Johnson's Champú fuerza vitamina	Jumbo	400	mL	30.950	7.737,5	1
P02	Johnson's Champú fuerza vitamina	Éxito	400	mL	33.100	8.275,0	2
P02	Johnson's Champú fuerza vitamina	Carulla	400	mL	34.400	8.600,0	3
P03	Arrurru Champú cabello claro	Éxito	750	mL	41.100	5.480,0	1
P03	Arrurru Champú cabello claro	Jumbo	750	mL	41.190	5.492,0	2
P03	Arrurru Champú cabello claro	Carulla	750	mL	42.500	5.666,7	3
P04	Johnson's Champú brillo gotas	Carulla	400	mL	21.840	5.460,0	1
P04	Johnson's Champú brillo gotas	Éxito	400	mL	22.680	5.670,0	2
P04	Johnson's Champú brillo gotas	Jumbo	400	mL	29.880	7.470,0	3
P05	Johnson's Champú hidratacion intensa	Éxito	400	mL	25.950	6.487,5	1
P05	Johnson's Champú hidratacion intensa	Carulla	400	mL	27.300	6.825,0	2
P05	Johnson's Champú hidratacion intensa	Jumbo	400	mL	29.430	7.357,5	3
P06	Johnson's Champú original	Éxito	750	mL	32.160	4.288,0	1
P06	Johnson's Champú original	Carulla	400	mL	24.800	6.200,0	2
P06	Johnson's Champú original	Jumbo	400	mL	32.220	8.055,0	3
P07	Arrurru Colonia arruru azul original	Éxito	220	mL	36.300	16.500,0	1
P07	Arrurru Colonia arruru azul original	Carulla	220	mL	36.700	16.681,8	2
P07	Arrurru Colonia arruru azul original	Jumbo	120	mL	27.790	23.158,3	3
P08	Johnson's Champú original	Éxito	750	mL	32.160	4.288,0	1
P08	Johnson's Champú original	Carulla	200	mL	19.700	9.850,0	2
P08	Johnson's Champú original	Jumbo	200	mL	21.640	10.820,0	3
P09	Johnson's Jabón avena tripack	Jumbo	108.900	g	16.020	14,7	1
P09	Johnson's Jabón avena tripack	Carulla	660	g	18.240	2.763,6	2
P09	Johnson's Jabón avena tripack	Éxito	660	g	18.240	2.763,6	2
P10	Arrurru Colonia arruru original rosa	Carulla	220	mL	35.500	16.136,4	1
P10	Arrurru Colonia arruru original rosa	Éxito	220	mL	35.500	16.136,4	1
P10	Arrurru Colonia arruru original rosa	Jumbo	120	mL	27.790	23.158,3	3
P11	Johnson's Champú cabello claro manzanilla	Carulla	400	mL	21.840	5.460,0	1
P11	Johnson's Champú cabello claro manzanilla	Éxito	400	mL	24.560	6.140,0	2
P11	Johnson's Champú cabello claro manzanilla	Jumbo	400	mL	30.950	7.737,5	3
P12	Baby Soft Champú docil nudos	Éxito	750	mL	31.050	4.140,0	1
P12	Baby Soft Champú docil nudos	Carulla	750	mL	31.400	4.186,7	2
P12	Baby Soft Champú docil nudos	Jumbo	750	mL	32.990	4.398,7	3

```
mejor <- q5 |> filter(puesto == 1)
tabla(mejor |> select(id, producto, comercio, contenido, unidad, precio, precio_x100),
      cap = "Presentación y comercio con menor precio por 100 mL/g en cada producto.",
      fs = 7.5, wrap = c(producto = "5cm"))
```

Tabla 34: Presentación y comercio con menor precio por 100 mL/g en cada producto.

id	producto	comercio	contenido	unidad	precio	precio_x100
P01	Johnson's Jabón cabeza pies	Carulla	400	mL	29.520	7.380,0
P01	Johnson's Jabón cabeza pies	Éxito	400	mL	29.520	7.380,0
P02	Johnson's Champú fuerza vitamina	Jumbo	400	mL	30.950	7.737,5
P03	Arrurru Champú cabello claro	Éxito	750	mL	41.100	5.480,0
P04	Johnson's Champú brillo gotas	Carulla	400	mL	21.840	5.460,0
P05	Johnson's Champú hidratacion intensa	Éxito	400	mL	25.950	6.487,5
P06	Johnson's Champú original	Éxito	750	mL	32.160	4.288,0

P07	Arrurru Colonia arruru azul original	Éxito	220	mL	36.300	16.500,0
P08	Johnson's Champú original	Éxito	750	mL	32.160	4.288,0
P09	Johnson's Jabón avena tripack	Jumbo	108.900	g	16.020	14,7
P10	Arrurru Colonia arruru original rosa	Carulla	220	mL	35.500	16.136,4
P10	Arrurru Colonia arruru original rosa	Éxito	220	mL	35.500	16.136,4
P11	Johnson's Champú cabello claro manzanilla	Carulla	400	mL	21.840	5.460,0
P12	Baby Soft Champú docil nudos	Éxito	750	mL	31.050	4.140,0

Interpretación. Cuando un producto aparece en tamaños distintos, el menor precio por unidad no siempre corresponde al empaque más barato: hay que separar el *desembolso* (precio del empaque) del *costo por 100 mL/g*. Los productos en mL y en g se rankean por separado.

5.5.1 Tamaños distintos: ahorro unitario frente a desembolso

Para el mismo producto vendido en tamaños diferentes se calcula el precio por unidad común (precio del empaque ÷ contenido total × 100) y se comparan **dos clasificaciones**: la del precio del empaque (cuánto se desembolsa) y la del precio por 100 mL/g (cuánto cuesta cada unidad de contenido).

```
sql5c <- paste0(SQL_BASE, "
SELECT c.producto_id AS id, c.nombre_comun AS producto, c.comercio,
       c.contenido_total AS contenido, c.unidad, c.precio_cop AS precio,
       ROUND(c.precio_100, 1) AS x100,
       RANK() OVER (PARTITION BY c.producto_id ORDER BY c.precio_cop)            AS puesto_desembolso,
       RANK() OVER (PARTITION BY c.producto_id ORDER BY ROUND(c.precio_100, 2))  AS puesto_x100
FROM c JOIN t ON t.producto_id = c.producto_id
WHERE t.cont_max * 1.0 / t.cont_min > 1.03
ORDER BY c.producto_id, puesto_x100, c.comercio")
q5c <- qq(sql5c)
if (nrow(q5c) > 0) {
  tabla(q5c, cap = "Productos con tamaños distintos: desembolso y precio por 100 mL/g (puesto 1 = más bar
        fs = 7.5, wrap = c(producto = "4.5cm"))
}
```

Tabla 35: Productos con tamaños distintos: desembolso y precio por 100 mL/g (puesto 1 = más barato).

id	producto	comercio	contenido	unidad	precio	x100	puesto_desembolso	puesto_x100
P06	Johnson's Champú original	Éxito	750	mL	32.160	4.288,0	2	1
P06	Johnson's Champú original	Carulla	400	mL	24.800	6.200,0	1	2
P06	Johnson's Champú original	Jumbo	400	mL	32.220	8.055,0	3	3
P07	Arrurru Colonia arruru azul original	Éxito	220	mL	36.300	16.500,0	2	1
P07	Arrurru Colonia arruru azul original	Carulla	220	mL	36.700	16.681,8	3	2
P07	Arrurru Colonia arruru azul original	Jumbo	120	mL	27.790	23.158,3	1	3
P08	Johnson's Champú original	Éxito	750	mL	32.160	4.288,0	3	1
P08	Johnson's Champú original	Carulla	200	mL	19.700	9.850,0	1	2
P08	Johnson's Champú original	Jumbo	200	mL	21.640	10.820,0	2	3
P09	Johnson's Jabón avena tripack	Jumbo	108.900	g	16.020	14,7	1	1
P09	Johnson's Jabón avena tripack	Carulla	660	g	18.240	2.763,6	2	2
P09	Johnson's Jabón avena tripack	Éxito	660	g	18.240	2.763,6	2	2
P10	Arrurru Colonia arruru original rosa	Carulla	220	mL	35.500	16.136,4	2	1
P10	Arrurru Colonia arruru original rosa	Éxito	220	mL	35.500	16.136,4	2	1
P10	Arrurru Colonia arruru original rosa	Jumbo	120	mL	27.790	23.158,3	1	3

```r
if (nrow(q5c) > 0) {
  res5c <- q5c |> group_by(id, producto) |>
    summarise(
      menor_desembolso = paste(comercio[puesto_desembolso == 1], collapse = " = "),
      cont_menor_desembolso = contenido[puesto_desembolso == 1][1],
      menor_x100 = paste(comercio[puesto_x100 == 1], collapse = " = "),
      cont_menor_x100 = contenido[puesto_x100 == 1][1],
      mayor_contenido = max(contenido), .groups = "drop") |>
    mutate(coinciden = ifelse(menor_desembolso == menor_x100, "sí", "no"),
           x100_en_el_mayor = ifelse(cont_menor_x100 == mayor_contenido, "sí", "no"))
  tabla(res5c |> rename(`menor desembolso` = menor_desembolso,
                        `cont. menor desembolso` = cont_menor_desembolso,
                        `menor x100` = menor_x100,
                        `cont. menor x100` = cont_menor_x100,
                        `mayor contenido` = mayor_contenido,
                        `x100 en el mayor` = x100_en_el_mayor),
        cap = "Contraste: comercio de menor desembolso frente al de menor precio por 100 mL/g.",
        fs = 7, wrap = c(producto = "3.6cm"))
}
```

Tabla 36: Contraste: comercio de menor desembolso frente al de menor precio por 100 mL/g.

id	producto	menor desembolso	cont. menor desembolso	menor x100	cont. menor x100	mayor contenido	coinciden	x100 en e
P06	Johnson's Champú original	Carulla	400	Éxito	750	750	no	sí
P07	Arrurru Colonia arruru azul original	Jumbo	120	Éxito	220	220	no	sí
P08	Johnson's Champú original	Carulla	200	Éxito	750	750	no	sí
P09	Johnson's Jabón avena tripack	Jumbo	108.900	Jumbo	108.900	108.900	sí	sí
P10	Arrurru Colonia arruru original rosa	Jumbo	120	Carulla = Éxito	220	220	no	sí

```r
ej_div <- if (nrow(q5c) > 0) res5c |> filter(coinciden == "no") |> slice(1) else tibble()
if (nrow(ej_div) > 0) {
  tabla(q5c |> filter(id == ej_div$id) |>
          select(comercio, contenido, unidad, precio, x100, puesto_desembolso, puesto_x100),
        cap = paste0("Ejemplo: ", ej_div$producto,
                     ". El menor desembolso y el menor costo por unidad no coinciden."),
        fs = 8)
}
```

Tabla 37: Ejemplo: Johnson's Champú original. El menor desembolso y el menor costo por unidad no coinciden.

comercio	contenido	unidad	precio	x100	puesto_desembolso	puesto_x100
Éxito	750	mL	32.160	4.288	2	1
Carulla	400	mL	24.800	6.200	1	2
Jumbo	400	mL	32.220	8.055	3	3

Interpretación. De los 5 productos con tamaños distintos, en 4 el comercio de menor desembolso no es el de menor precio por 100 mL/g, y en 5 el menor costo por unidad correspondió al empaque más grande. Un envase mayor suele costar menos por 100 mL/g pero exige un desembolso mayor; ambas conclusiones son válidas y responden a preguntas distintas (qué conviene para un presupuesto ajustado frente a qué conviene para un consumo recurrente). Los productos en mL y en g no se mezclan en un mismo ranking, y el contenido de los paquetes múltiples se toma como el total (2×200 mL $= 400$ mL). Además, solo se compara dentro del mismo producto y formulación: pertenecer a la misma categoría no hace equivalentes a todos los productos.

5.6 Pregunta 6 – Mayores diferencias entre comercios

```
sql6 <- paste0(SQL_BASE, "
SELECT producto_id AS id, nombre_comun AS producto, unidad, COUNT(DISTINCT comercio) AS n_com,
       ROUND(MIN(precio_100), 1) AS x100_min, ROUND(MAX(precio_100), 1) AS x100_max,
       ROUND(MAX(precio_100) - MIN(precio_100), 1) AS ahorro_x100,
       ROUND(100.0 * (MAX(precio_100) - MIN(precio_100)) / MAX(precio_100), 1) AS ahorro_pct,
       CASE WHEN MAX(contenido_total) * 1.0 / MIN(contenido_total) <= 1.03
            THEN 'sí' ELSE 'no' END AS misma_presentacion
FROM c GROUP BY producto_id, nombre_comun, unidad
ORDER BY ahorro_pct DESC")
q6 <- qq(sql6)
tabla(q6 |> rename(`misma presentación` = misma_presentacion),
      cap = "Pregunta 6: presentaciones ordenadas por ahorro porcentual (sobre precio por 100 mL/g).",
      fs = 7.5, wrap = c(producto = "4cm", `misma presentación` = "1.8cm"))
```

Tabla 38: Pregunta 6: presentaciones ordenadas por ahorro porcentual (sobre precio por 100 mL/g).

id	producto	unidad	n_com	x100_min	x100_max	ahorro_x100	ahorro_pct	misma presentación
P09	Johnson's Jabón avena tripack	g	3	14,7	2.763,6	2.748,9	99,5	no
P08	Johnson's Champú original	mL	3	4.288,0	10.820,0	6.532,0	60,4	no
P06	Johnson's Champú original	mL	3	4.288,0	8.055,0	3.767,0	46,8	no
P10	Arrurru Colonia arruru original rosa	mL	3	16.136,4	23.158,3	7.022,0	30,3	no
P11	Johnson's Champú cabello claro manzanilla	mL	3	5.460,0	7.737,5	2.277,5	29,4	sí
P07	Arrurru Colonia arruru azul original	mL	3	16.500,0	23.158,3	6.658,3	28,8	no
P04	Johnson's Champú brillo gotas	mL	3	5.460,0	7.470,0	2.010,0	26,9	sí
P01	Johnson's Jabón cabeza pies	mL	3	7.380,0	9.572,5	2.192,5	22,9	sí
P05	Johnson's Champú hidratacion intensa	mL	3	6.487,5	7.357,5	870,0	11,8	sí
P02	Johnson's Champú fuerza vitamina	mL	3	7.737,5	8.600,0	862,5	10,0	sí
P12	Baby Soft Champú docil nudos	mL	3	4.140,0	4.398,7	258,7	5,9	sí
P03	Arrurru Champú cabello claro	mL	3	5.480,0	5.666,7	186,7	3,3	sí

Interpretación. La mayor diferencia es 99.5 % (Johnson's Jabón avena tripack). Si `misma_presentacion = no`, parte de la diferencia refleja el tamaño del empaque y no solo el comercio.

5.7 Pregunta 7 – Datos faltantes o inconsistentes

```
sql7 <- "
SELECT comercio, COUNT(*) AS incluidos,
       SUM(precio_cop IS NULL)       AS sin_precio,
       SUM(marca_std IS NULL)        AS sin_marca,
       SUM(contenido_total IS NULL)  AS sin_contenido,
       SUM(sku IS NULL)              AS sin_sku,
       SUM(es_promocion = 1)         AS en_promocion,
       SUM(vendedor_externo = 1)     AS vendedor_externo,
       SUM(precio_atipico = 1)       AS precio_atipico
FROM observaciones WHERE ejecucion_id = '@EJ@' AND incluido = 1
GROUP BY comercio ORDER BY comercio"
```

```
q7 <- qq(sql7)
tabla(q7 |> rename_with(~ gsub("_", " ", .x)),
      cap = "Pregunta 7: faltantes e inconsistencias en productos incluidos.", fs = 7)
```

Tabla 39: Pregunta 7: faltantes e inconsistencias en productos incluidos.

comercio	incluidos	sin precio	sin marca	sin contenido	sin sku	en promocion	vendedor externo	precio atipico
Carulla	85	0	4	3	12	0	11	2
Jumbo	30	0	0	0	12	2	0	0
Éxito	73	0	6	7	29	0	11	0

```
sql7b <- "
SELECT comercio, COUNT(*) AS obs_comparables,
       SUM(es_promocion = 1) AS en_promocion, SUM(precio_atipico = 1) AS atipicos,
       SUM(contenido_total IS NULL) AS sin_contenido
FROM observaciones WHERE comparable = 1 AND ejecucion_id = '@EJ@'
GROUP BY comercio"
q7b <- qq(sql7b)
tabla(q7b, cap = "Pregunta 7: problemas dentro de los productos comparables.")
```

Tabla 40: Pregunta 7: problemas dentro de los productos comparables.

comercio	obs_comparables	en_promocion	atipicos	sin_contenido
Carulla	12	0	0	0
Jumbo	12	0	0	0
Éxito	12	0	0	0

Interpretación. `sin_contenido` impide calcular el precio por 100 mL/g y deja al producto fuera del emparejamiento; `sin_marca` lo excluye también. `en_promocion` indica que el precio observado es promocional (válido solo en la fecha de captura) y `precio_atipico` señala valores fuera de 0,6×–1,6× la mediana de productos equivalentes, que conviene revisar. Dentro de los comparables hay 0 observaciones en promoción y 0 atípicas.

5.8 Pregunta 8 – Oferta por comercio y calidad de la información

```
sql8 <- "
SELECT ejecucion_id, comercio,
       COUNT(*)                          AS obtenidos,
       SUM(incluido = 1)                 AS incluidos,
       ROUND(100.0 * SUM(incluido = 1 AND nombre_original IS NOT NULL) / SUM(incluido = 1), 1) AS pct_des
       ROUND(100.0 * SUM(incluido = 1 AND marca_std IS NOT NULL)       / SUM(incluido = 1), 1) AS pct_mar
       ROUND(100.0 * SUM(incluido = 1 AND contenido_total IS NOT NULL) / SUM(incluido = 1), 1) AS pct_con
FROM observaciones GROUP BY ejecucion_id, comercio ORDER BY ejecucion_id DESC, comercio"
q8 <- q(sql8)
tabla(q8, cap = "Pregunta 8: tamaño de la oferta y porcentaje de productos incluidos con descripción, mar
      fs = 7.5)
```

Tabla 41: Pregunta 8: tamaño de la oferta y porcentaje de productos incluidos con descripción, marca y contenido identificables.

ejecucion_id	comercio	obtenidos	incluidos	pct_descripcion	pct_marca	pct_contenido

2026-10-08T225121	Carulla	104	84	100	95,2	96,4
2026-10-08T225121	Jumbo	104	83	100	100,0	97,6
2026-10-08T225121	Éxito	326	69	100	91,3	89,9
2026-10-07T110040	Carulla	105	85	100	95,3	96,5
2026-10-07T110040	Jumbo	104	30	100	100,0	100,0
2026-10-07T110040	Éxito	451	73	100	91,8	90,4
2026-10-07T101436	Carulla	104	78	100	47,4	96,2
2026-10-07T101436	Jumbo	62	39	0	87,2	100,0
2026-10-07T101436	Éxito	451	73	100	24,7	90,4

```
oferta <- qq("SELECT comercio, subcategoria_std AS subcategoria, COUNT(*) AS productos
              FROM observaciones WHERE ejecucion_id = '@EJ@' AND incluido = 1
              GROUP BY comercio, subcategoria_std")
tabla(oferta |> pivot_wider(names_from = comercio, values_from = productos, values_fill = 0),
      cap = "Pregunta 8: productos incluidos por subcategoría y comercio.")
```

Tabla 42: Pregunta 8: productos incluidos por subcategoría y comercio.

subcategoria	Carulla	Jumbo	Éxito
Aceite	6	0	4
Champú	32	10	34
Colonia	12	2	15
Crema hidratante	20	0	11
Jabón	15	18	8
Loción	0	0	1

Interpretación. El comercio con mayor oferta incluida en la ejecución analizada es **Carulla**. Los porcentajes de marca y contenido identificables indican qué tan confiable es el cálculo del precio por 100 mL/g en cada sitio.

6 Visualizaciones

```
comp <- qq("SELECT o.producto_id, p.nombre_comun, o.comercio, o.contenido_total, o.unidad,
                   o.precio_cop, o.precio_antes_cop, o.es_promocion, o.precio_100
            FROM observaciones o JOIN productos p ON p.producto_id = o.producto_id
            WHERE o.comparable = 1 AND o.ejecucion_id = '@EJ@' AND o.precio_cop IS NOT NULL") |>
  group_by(producto_id) |>
  mutate(cont_ref = min(contenido_total), ratio = max(contenido_total) / min(contenido_total),
         misma = ratio <= 1.03,
         etq = paste0(producto_id, " · ", str_trunc(nombre_comun, 30))) |>
  ungroup() |>
  mutate(comercio = factor(comercio, levels = names(COL)))
```

6.1 Precio del empaque en presentaciones idénticas

```
d1 <- comp |> filter(misma) |>
  mutate(etq = paste0(etq, " (", round(cont_ref), " ", unidad, ")"))
if (nrow(d1) > 0) {
  ggplot(d1, aes(x = precio_cop, y = fct_rev(etq), fill = comercio)) +
    geom_col(position = position_dodge(width = 0.8), width = 0.75) +
    scale_fill_manual(values = COL, name = "Comercio", drop = FALSE) +
```

```
    scale_x_continuous(labels = cop, expand = expansion(mult = c(0, 0.08))) +
    labs(x = "Precio del empaque (COP)", y = NULL) +
    theme_minimal(base_size = 9) + theme(legend.position = "bottom")
} else cat("No hay presentaciones idénticas entre los tres comercios.\n")
```

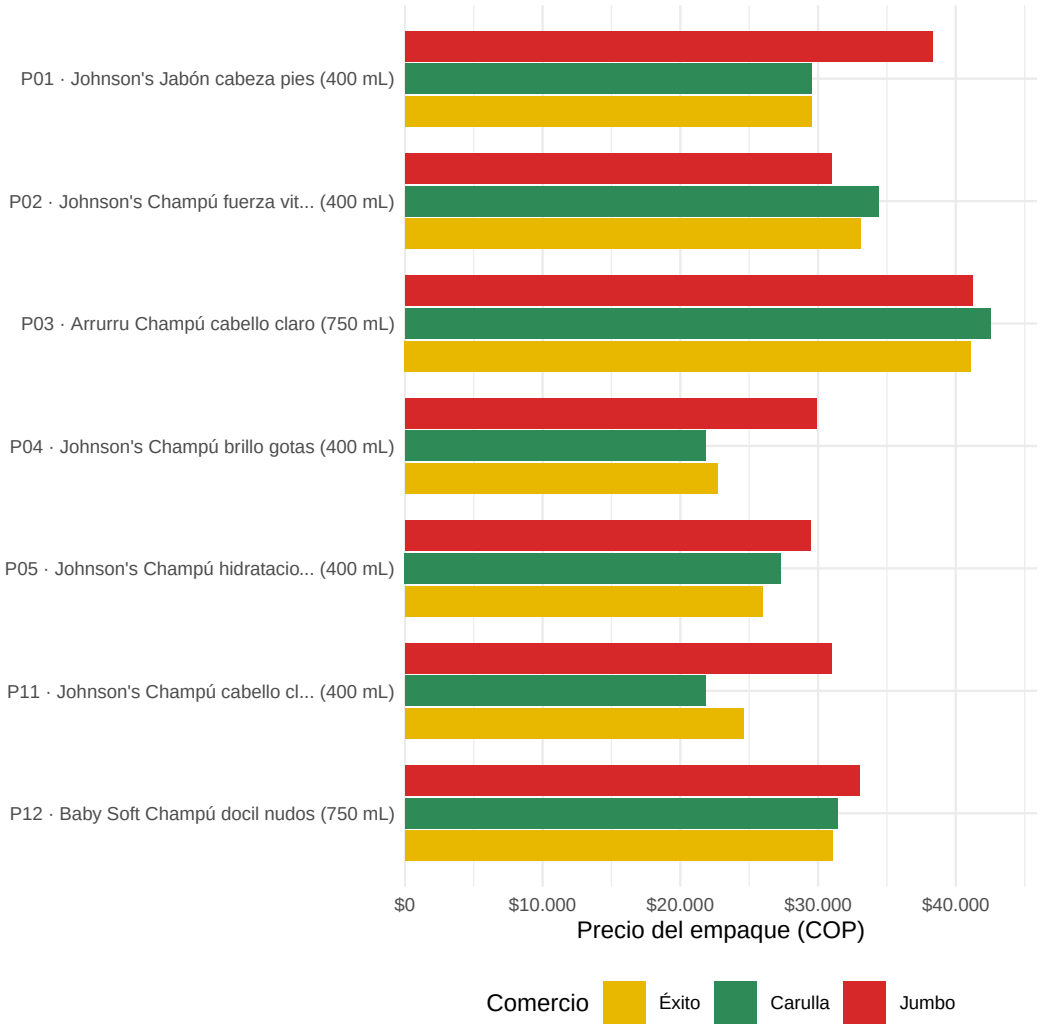


Figura 1: Precio del empaque (COP) por comercio. Solo presentaciones equivalentes (contenido ±3 %). Misma ejecución.

Interpretación. Para cada producto, la barra más corta indica el comercio con menor desembolso. Barras casi iguales indican empate práctico. **Limitación:** son precios del momento de captura y pueden incluir promociones.

6.2 Precio por 100 mL / 100 g

```
d2 <- comp |> mutate(etq_p = paste0(producto_id, " · ", str_trunc(nombre_comun, 30)))
ggplot(d2, aes(x = precio_100, y = fct_rev(etq_p))) +
  geom_line(aes(group = etq_p), colour = "grey70") +
  geom_point(aes(colour = comercio, shape = misma), size = 3) +
```

```
  facet_grid(unidad ~ ., scales = "free_y", space = "free_y",
             labeller = labeller(unidad = c(mL = "por 100 mL", g = "por 100 g"))) +
  scale_colour_manual(values = COL, name = "Comercio", drop = FALSE) +
  scale_shape_manual(values = c(`TRUE` = 16, `FALSE` = 17),
                     labels = c(`TRUE` = "mismo tamaño", `FALSE` = "distinto tamaño"), name = NULL) +
  scale_x_continuous(labels = cop) +
  labs(x = "COP por 100 mL (o 100 g, según el panel)", y = NULL) +
  theme_minimal(base_size = 9) + theme(legend.position = "bottom")
```

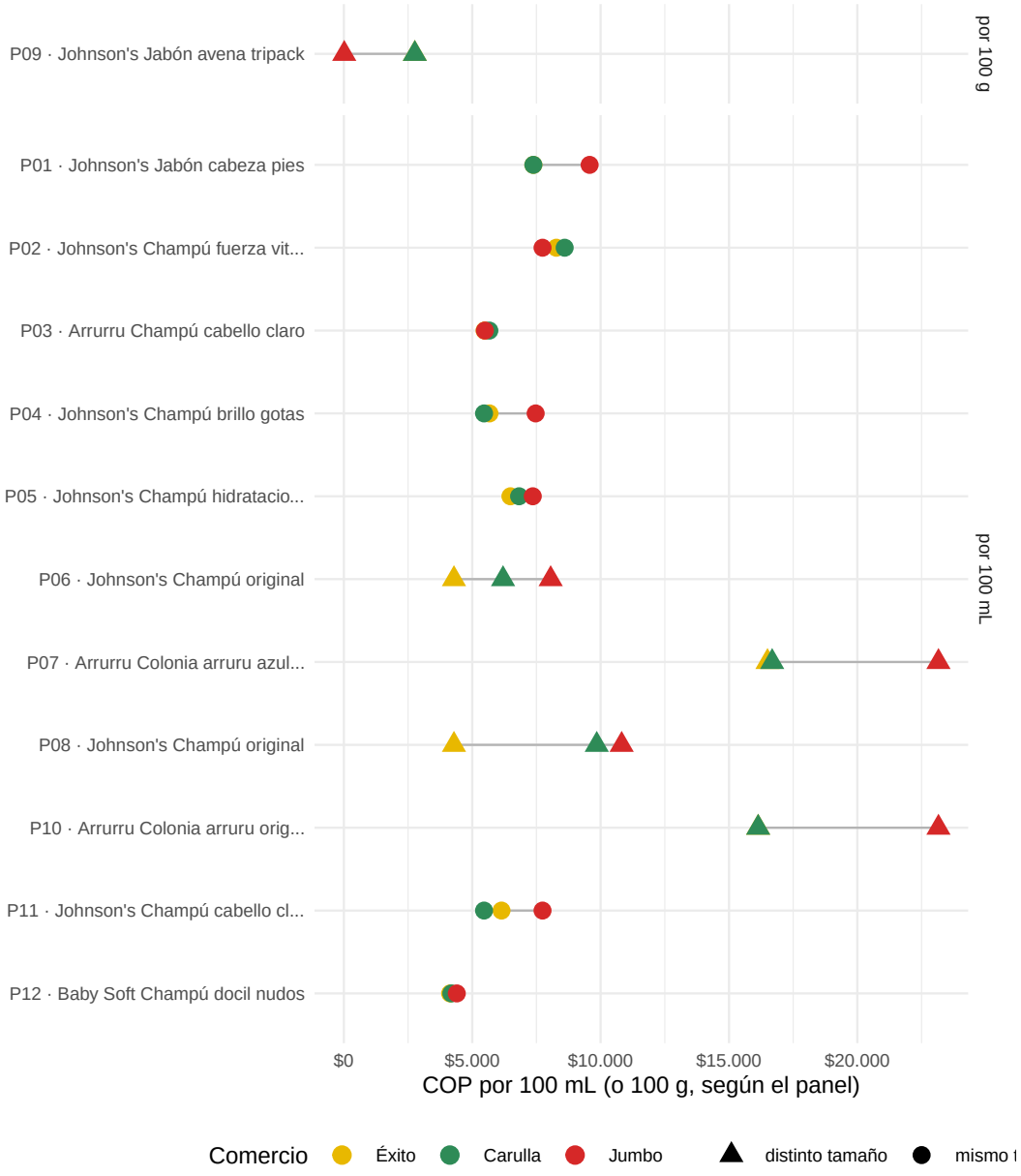


Figura 2: Precio por 100 mL o 100 g (COP) por producto y comercio; cada panel usa una sola unidad.

Interpretación. Este gráfico es el correcto para comparar tamaños distintos. Los triángulos señalan productos cuyos comercios venden tamaños diferentes: allí parte de la diferencia proviene del tamaño del empaque (los

envases grandes suelen tener menor costo por 100 mL/g, pero exigen un mayor desembolso). Los paneles no se mezclan porque mL y g son magnitudes distintas.

6.3 Oportunidades de ahorro

```
d3 <- comp |> group_by(producto_id, etq) |>
  summarise(pmax_ = max(precio_100), pmin_ = min(precio_100),
            barato = paste(comercio[precio_100 == min(precio_100)][1]),
            misma = first(misma), .groups = "drop") |>
  mutate(pct = 100 * (pmax_ - pmin_) / pmax_)
ggplot(d3, aes(x = pct, y = fct_reorder(etq, pct), fill = barato, alpha = misma)) +
  geom_col() +
  geom_text(aes(label = paste0(round(pct, 1), " %")), hjust = -0.1, size = 2.8, alpha = 1) +
  scale_fill_manual(values = COL, name = "Más barato", drop = FALSE) +
  scale_alpha_manual(values = c(`TRUE` = 1, `FALSE` = 0.55),
                     labels = c(`TRUE` = "mismo tamaño", `FALSE` = "distinto tamaño"), name = NULL) +
  scale_x_continuous(expand = expansion(mult = c(0, 0.15))) +
  labs(x = "Ahorro (%) sobre el precio por 100 mL/g más alto", y = NULL) +
  theme_minimal(base_size = 9) + theme(legend.position = "bottom")
```

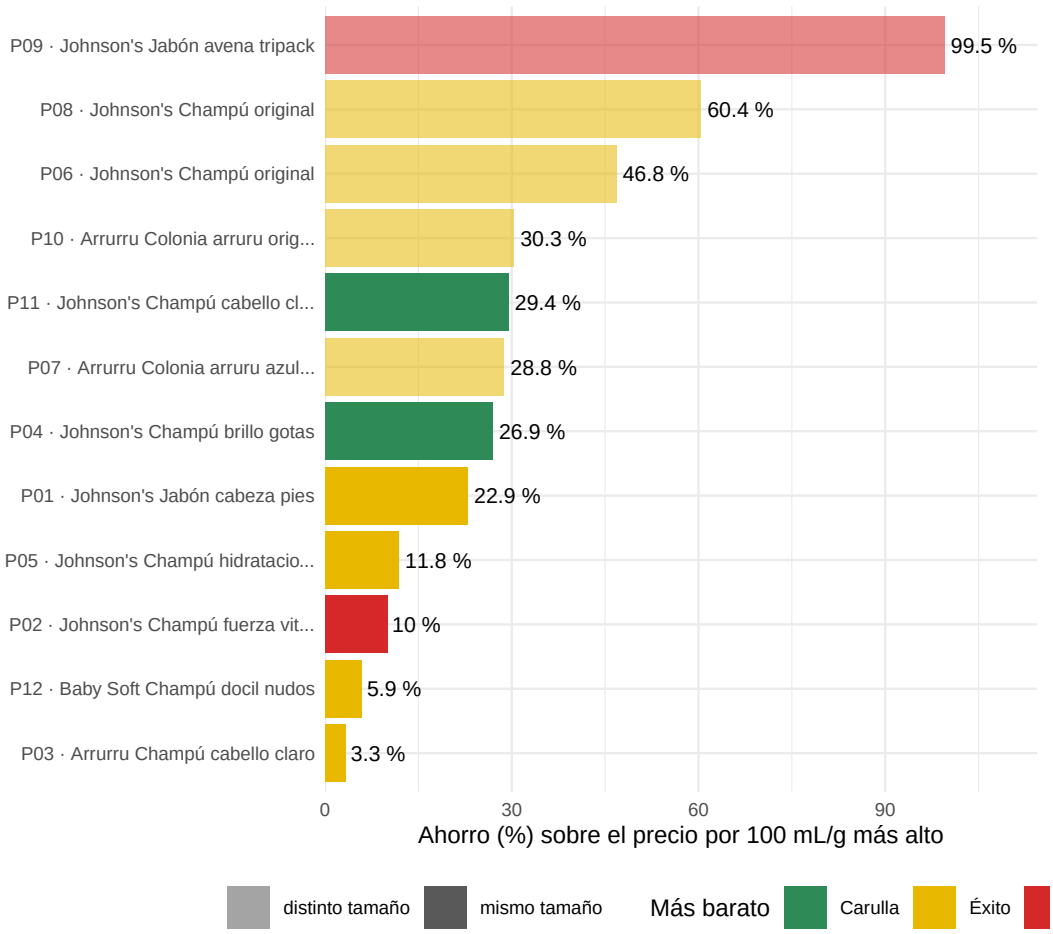


Figura 3: Ahorro porcentual entre el precio por 100 mL/g más alto y el más bajo de cada producto; color = comercio más barato.

Interpretación. Las barras largas son los productos donde elegir bien el comercio rinde más. Las barras semi-transparentes mezclan efecto-tamaño con efecto-comercio. **Limitación:** el ahorro se calcula con un solo corte de precios.

6.4 Canasta con presentaciones idénticas

```
canasta <- comp |> filter(misma) |> group_by(producto_id) |>
  filter(n_distinct(comercio) == 3) |> ungroup()
n_can <- n_distinct(canasta$producto_id)
tot <- canasta |> group_by(comercio) |>
  summarise(total = sum(precio_cop), .groups = "drop") |>
  mutate(`diferencia vs. más barato` = total - min(total))
```

```
if (n_can > 0) {
  tabla(tot, cap = paste0("Canasta de ", n_can, " presentaciones idénticas (COP)."))
} else {
  cat("No hay presentaciones idénticas completas para armar una canasta.\n")
}
```

Tabla 43: Canasta de 7 presentaciones idénticas (COP).

comercio	total	diferencia vs. más barato
Éxito	207.960	0
Carulla	208.800	840
Jumbo	233.680	25.720

```
if (n_can > 0) {
  ggplot(tot, aes(x = comercio, y = total, fill = comercio)) +
    geom_col(width = 0.6, show.legend = FALSE) +
    geom_text(aes(label = cop(total)), vjust = -0.4, size = 3) +
    scale_fill_manual(values = COL) +
    scale_y_continuous(labels = cop, expand = expansion(mult = c(0, 0.15))) +
    labs(x = NULL, y = "Total canasta (COP)") + theme_minimal(base_size = 9)
}
```

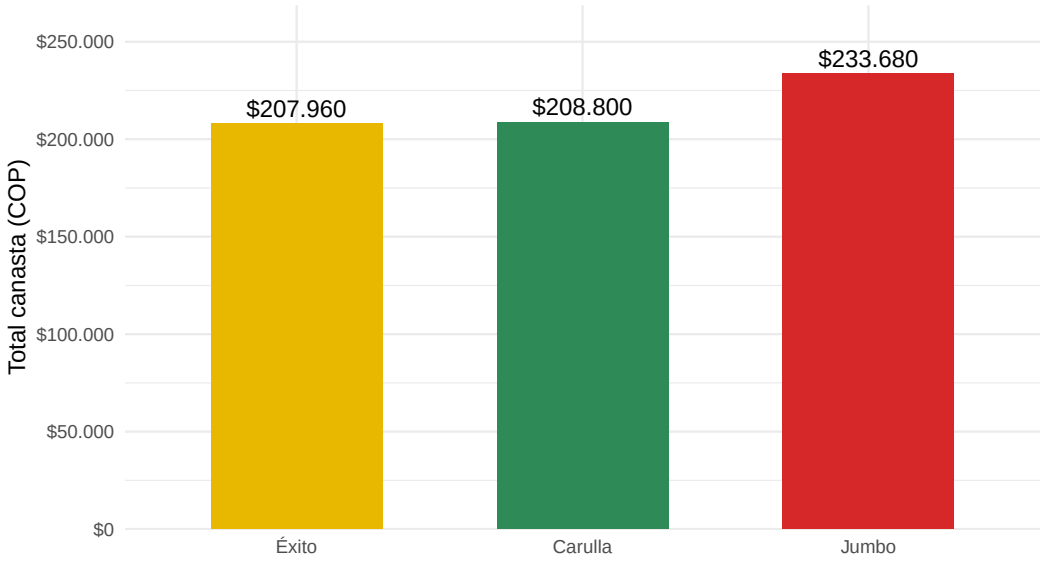


Figura 4: Costo total de la canasta formada por las mismas presentaciones y cantidades en los tres comercios (una unidad de cada una).

Interpretación. Solo es válido comparar canastas con exactamente las mismas presentaciones y cantidades, con cobertura completa y precios de una misma ejecución; por eso se usan únicamente los productos con presentación idéntica en los tres comercios. No debe leerse como que un comercio es «globalmente» más barato. Entre la canasta más cara y la más barata hay un ahorro de 25.720 COP (11 %), con Ahorro (COP) = Pmax − Pmin y Ahorro (%) = 100 (Pmax − Pmin) / Pmax, válido solo para estos 7 productos.

6.5 Oferta por comercio

```
ggplot(oferta |> mutate(comercio = factor(comercio, levels = names(COL))),
       aes(x = fct_reorder(subcategoria, productos, .fun = sum), y = productos, fill = comercio)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.75) +
  coord_flip() +
  scale_fill_manual(values = COL, name = "Comercio", drop = FALSE) +
  labs(x = NULL, y = "Número de productos") +
  theme_minimal(base_size = 9) + theme(legend.position = "bottom")
```

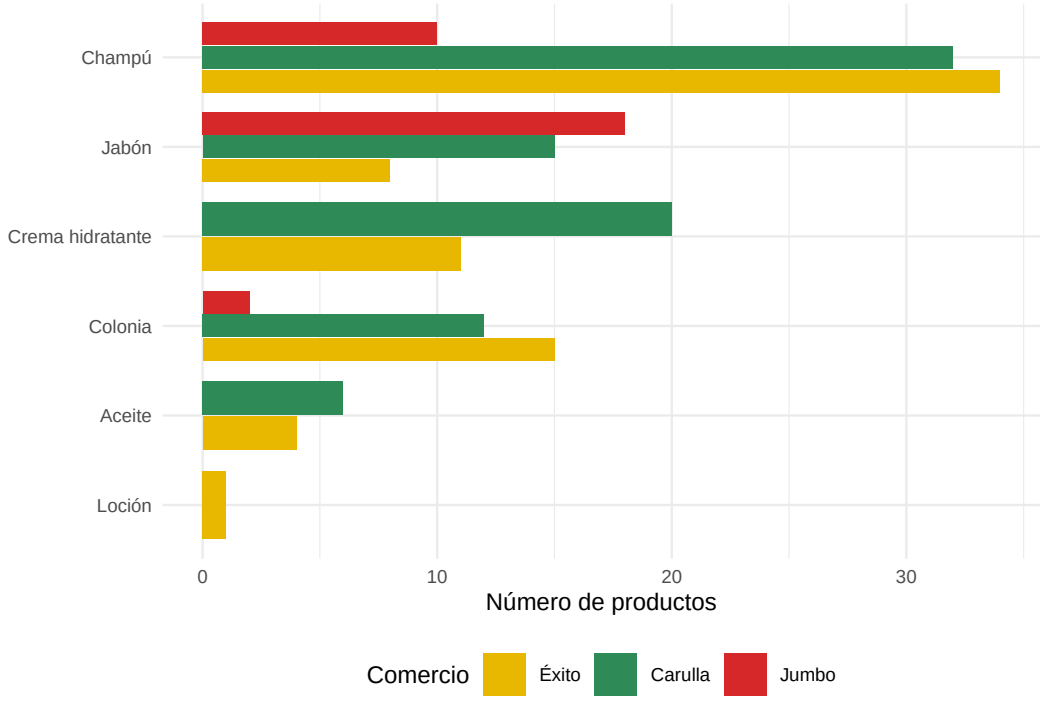


Figura 5: Productos incluidos en la categoría por subcategoría y comercio (ejecución analizada).

Interpretación. Describe el surtido de cada comercio en la categoría. Un surtido mayor no implica precios menores: el tamaño de la oferta y el nivel de precios son cosas distintas.

6.6 Cambios entre ejecuciones

```
hist <- q("SELECT o.ejecucion_id, o.producto_id, o.comercio, o.precio_100, o.precio_cop
           FROM observaciones o WHERE o.comparable = 1 AND o.precio_100 IS NOT NULL")
n_ej <- n_distinct(hist$ejecucion_id)
if (n_ej >= 2) {
  hist |> mutate(comercio = factor(comercio, levels = names(COL))) |>
    ggplot(aes(x = ejecucion_id, y = precio_100, group = comercio, colour = comercio)) +
    stat_summary(fun = median, geom = "line") + stat_summary(fun = median, geom = "point") +
    scale_colour_manual(values = COL, name = "Comercio") + scale_y_continuous(labels = cop) +
    labs(x = "Ejecución", y = "Mediana de COP por 100 mL/g") +
    theme_minimal(base_size = 9) + theme(axis.text.x = element_text(angle = 30, hjust = 1))
} else cat("Solo hay una ejecución con comparables; con una segunda ejecución en otra fecha aquí se vería
```

```
Solo hay una ejecución con comparables; con una segunda ejecución en otra fecha aquí se vería la evolució
```

7 Limitaciones y conclusiones

7.1 Guía de compra según los datos observados

```
guia <- comp |> group_by(producto_id) |>
  summarise(
    producto = first(nombre_comun), unidad = first(unidad),
    mas_barato_x100 = paste(comercio[precio_100 == min(precio_100)], collapse = " = "),
    x100_min = round(min(precio_100)),
    menor_desembolso = paste(comercio[precio_cop == min(precio_cop)], collapse = " = "),
    desembolso_min = min(precio_cop),
    tamano = if (first(misma)) "igual" else
      if (contenido_total[which.min(precio_100)] == max(contenido_total)) "el mayor" else "no es el mayor
    .groups = "drop")
tabla(guia |> rename(id = producto_id,
                     `más barato x100` = mas_barato_x100,
                     `x100 mín.` = x100_min,
                     `menor desembolso` = menor_desembolso,
                     `desembolso mín.` = desembolso_min,
                     `tamaño del más barato x100` = tamano),
      cap = "Dónde conviene comprar cada producto. «x100» = COP por 100 mL/g; «desembolso» = precio del e
      fs = 7,
      wrap = c(producto = "3.2cm", `más barato x100` = "2.3cm",
               `menor desembolso` = "2.3cm", `tamaño del más barato x100` = "2.3cm"))
```

Tabla 44: Dónde conviene comprar cada producto. «x100» = COP por 100 mL/g; «desembolso» = precio del empaque.

id	producto	unidad	más barato x100	x100 mín.	menor desembolso	desembolso mín.	tamaño del más barato x100
P01	Johnson's Jabón cabeza pies	mL	Éxito = Carulla	7.380	Éxito = Carulla	29.520	igual
P02	Johnson's Champú fuerza vitamina	mL	Jumbo	7.738	Jumbo	30.950	igual
P03	Arrurru Champú cabello claro	mL	Éxito	5.480	Éxito	41.100	igual
P04	Johnson's Champú brillo gotas	mL	Carulla	5.460	Carulla	21.840	igual
P05	Johnson's Champú hidratacion intensa	mL	Éxito	6.488	Éxito	25.950	igual
P06	Johnson's Champú original	mL	Éxito	4.288	Carulla	24.800	el mayor
P07	Arrurru Colonia arruru azul original	mL	Éxito	16.500	Jumbo	27.790	el mayor
P08	Johnson's Champú original	mL	Éxito	4.288	Carulla	19.700	el mayor
P09	Johnson's Jabón avena tripack	g	Jumbo	15	Jumbo	16.020	el mayor
P10	Arrurru Colonia arruru original rosa	mL	Éxito = Carulla	16.136	Jumbo	27.790	el mayor
P11	Johnson's Champú cabello claro manzanilla	mL	Carulla	5.460	Carulla	21.840	igual
P12	Baby Soft Champú docil nudos	mL	Éxito	4.140	Éxito	31.050	igual

```
gana <- table(unlist(strsplit(guia$mas_barato_x100, " = ")))
gana <- sort(gana, decreasing = TRUE)
mayor_n <- sum(guia$tamano == "el mayor")
dist_n  <- sum(guia$tamano != "igual")
```

- **Dónde comprar.** El comercio con más productos al menor precio por 100 mL/g (contando empates) es **Éxito** (8 de 12 productos). Ninguno es más barato en todo: la elección depende del producto, como muestra la tabla anterior.

- **Presentación grande vs. pequeña.** En 5 productos con tamaños distintos, el menor costo por unidad correspondió al empaque más grande en 5. Un empaque grande puede reducir el costo por 100 mL/g, pero exige mayor desembolso inmediato; si no se va a consumir o el producto caduca antes, el ahorro unitario no se materializa.
- **Ahorro unitario vs. desembolso total.** El menor precio por empaque (columna `menor desembolso`) y el menor precio por unidad (`más barato x100`) no siempre coinciden. Para presupuestos ajustados puede convenir el menor desembolso; para compras recurrentes, el menor precio por unidad.
- **Condiciones de compra.** Los precios comparados son los precios públicos del momento de captura. No incluyen el costo de envío, los montos mínimos de compra, ni descuentos que dependen de la tarjeta del comercio (p. ej. Jumbo Más) o de promociones temporales (`es_promocion`); esas condiciones pueden cambiar cuál opción conviene en una compra concreta.

7.2 Limitaciones

- **Fotografía en el tiempo:** los precios corresponden a la fecha y hora de consulta; no son permanentes. Con una sola ejecución no se observan cambios de precio.
- **Promociones:** el precio extraído puede ser promocional (`es_promocion`); en Jumbo el precio con tarjeta Jumbo Más no se usa en la comparación.
- **Vendedores externos** (marketplace) se excluyen del emparejamiento para comparar solo oferta directa.
- **Correspondencia:** se hizo con reglas documentadas (marca, tipo, unidad, similitud de nombre y rango de tamaño). Los niveles B–D tienen menor certeza que la presentación idéntica (nivel A) y deben revisarse.
- **Completitud:** donde el sitio no muestra el total de resultados no se puede verificar la cobertura contra el sitio (ver tabla de cobertura).
- **Verificabilidad:** las copias de `raw/` permiten reproducir la extracción sin conexión, pero reflejan el HTML del momento de captura; si un sitio cambia su estructura, los selectores deben actualizarse para nuevas ejecuciones en línea.
- **Entrega mínima:** la cobertura obligatoria (8 productos $\times$ 3 comercios en una misma ejecución) se valida en la pregunta 2; si no se cumple, debe ampliarse `TERMINOS` o cambiarse alguno de los comercios.

8 Reproducibilidad

Instrucciones completas en `README.md`. Las dependencias con versiones fijas están en `renv.lock` (`renv::restore()`). Resumen:

```
# Recolección en línea y registro de comparables
source("Scraper.R")
ejecutar_recoleccion(modo = "online",  db = "comparador_precios.sqlite")
ejecutar_comparables(db = "comparador_precios.sqlite", n_obj = 12, registrar = TRUE)

# Reprocesar sin conexión desde raw/
ejecutar_recoleccion(modo = "offline", db = "comparador_precios.sqlite")
ejecutar_comparables(db = "comparador_precios.sqlite", n_obj = 12, registrar = TRUE)

# Generar este documento
quarto render taller_1.qmd   # requiere TinyTeX/LaTeX para el PDF
```